Py2HWSW

A Python Framework for HW/SW Co-design

September 3, 2026

Design Specification, 0.81.0 , Build 280592d3f

Contents

List of Tables

List of Figures

1 Introduction

Open-source Python framework for managing files, automating project flows of embedded hardware/software codesign projects, and partially generating Verilog hardware components. The framework simplifies the project structure, addresses challenges in Hardware Design Languages like Verilog and VHDL, and automates emulation, simulation, FPGA, and ASIC flows. The proposed Verilog generator offers flexibility, user control, and ease of use, producing human-readable code compatible across FPGAs and ASICs.

1.1 What Is Py2HWSW?

In the rapidly evolving landscape of hardware design, the need for efficient and flexible tools is paramount. Enter py2hwsw, a powerful tool designed to streamline the process of generating Verilog cores from high-level descriptions provided in Python or JSON dictionaries. With py2hwsw, engineers can easily translate their design specifications into functional hardware components, significantly reducing development time and complexity.

1.2 What Is Py2HWSW For?

Py2HWSW is designed to do the following:

- **Core Generation**: Generates Verilog cores from descriptions in Python or JSON dictionaries.
- **Framework Compatibility**: Integrates seamlessly with existing Verilog cores and frameworks.
- **High-Level Configuration**: Allows configuration of cores via high-level Python parameters.
- **Automated Resources**: Produces scripts and Makefiles for deployment in various FPGAs, simulators, and synthesis tools, along with documentation.
- **Readable Code**: Generates legible Verilog code with comments for better understanding and maintenance.

1.3 What Problem Does Py2HWSW Solve?

Py2hwsw addresses several key challenges in the hardware design process:

- **Complexity of Verilog Coding**: Writing Verilog code can be intricate and error-prone, especially for those who may not be deeply familiar with hardware description languages. Py2hwsw simplifies this by allowing designers to specify their hardware requirements using high-level Python or JSON dictionaries, reducing the need for extensive Verilog knowledge.
- **Integration of Existing Designs**: Many projects involve legacy Verilog cores that need to be integrated with new designs. Py2hwsw facilitates this integration, allowing users to provide existing components and still benefit from the remaining automation features.
- **Configuration Challenges**: Customizing hardware components often requires deep dives into low-level code. Py2hwsw allows for high-level configuration through Python parameters, making it easier for designers to adjust their designs without getting bogged down in the details of Verilog.

- **Resource Generation**: The process of preparing scripts and Makefiles for various deployment environments can be tedious and time-consuming. Py2hwsw automates this process, providing users with the necessary resources to run their designs on different FPGAs, simulators, and synthesis tools.
- **Code Readability and Maintenance**: Maintaining and debugging hardware designs can be challenging, especially when the code is not well-documented. Py2hwsw generates legible Verilog code with comments, enhancing readability and making it easier for teams to collaborate and maintain their designs over time.

In summary, Py2hwsw streamlines the hardware design workflow, making it more accessible, efficient, and manageable for engineers and designers.

1.4 What Design Principles Underlie Py2HWSW?

Py2HWSW works by:

- **Standard Py2HWSW syntax**: Use a standard Py2HWSW syntax to describe each core.
- **Support Python dictionaries and JSON files**: Supports Python dictionaries to generate dynamic cores based on python parameters. And supports JSON files to describe fixed cores and for compatibility with cores generated by external tools.
- **Support custom verilog snippets**: Each core may include custom verilog snippets for any edge-case which cannot be described using Py2HWSW syntax.
- **Internal Object-Oriented structure**: Py2HWSW converts core descriptions into its internal object-oriented system, creating high-level abstractions of Verilog building blocks.

1.5 How Does Py2HWSW Accomplish Its Goals?

- **Two-Step Development Process**: The core development is divided into two distinct phases: the **setup** phase and the **build** phase. During the setup phase, Verilog source files are generated based on high-level descriptions provided in Python or JSON format. The build phase then utilizes these Verilog sources to produce the necessary FPGA bitstreams, netlists, and other deployment files.
- **Independent Setup Folders**: Each core is organized within its own independent setup folder, containing high-level description files and, if needed, low-level files as well.
- **Core Description Input**: The core's specifications are provided to Py2hwsw in the form of JSON or a Python dictionary, utilizing standard Py2hwsw attributes.
- **Flexible Attribute Handling**: When generating the cores dictionary via a Python script, users can include a set of standard Py2hwsw attributes alongside their own custom-defined attributes.

2 Getting Started

2.1 Setup Directory

The setup directory of a core may have the following structure:


```
.
├── core_name.py
├── core_name.json
├── document
│   ├── doc_build.mk
│   ├── figures
│   └── tsrc
├── hardware
│   ├── src
│   ├── fpga
│   │   ├── fpga_build.mk
│   │   ├── src
│   │   ├── quartus
│   │   └── vivado
│   ├── modules
│   ├── simulation
│   │   ├── sim_build.mk
│   │   └── src
│   └── syn
│       ├── src
│       └── genus
├── software
│   ├── sw_build.mk
│   └── src
├── scripts
├── submodules
├── Makefile
├── README.md
├── LICENSE
├── CITATION.cff
└── default.nix
```

Only the `core_name.py` or `core_name.json` file is needed to pass the core's description to Py2HWSW. The remaining directories and files are optional.

If the `document`, `hardware`, and `software` directories exist, they will be copied to the `build` directory, overriding any files already present there, such as standard ones or files from other cores.

The `*_build.mk` files allow the user to include core specific Makefile targets and variables from the build process. These will be copied to the `build` directory and included in the standard build process Makefiles.

The `src` directories contain manually written Verilog/C/TeX sources for the core, should they be needed.

The following directories and files do not follow a mandatory structure, but are typically used for the following purposes:

The `hardware/modules` and `submodules` directories typically contain setup directories of other cores.

The `scripts` directory contains scripts specific to the core, and may be called by the user or from the `core_name.py` script.

A simple example of a core's setup directory is available for the iob_and core.

A more complex example of a core's setup directory is available for the iob_soc core.

2.2 Create An AND Gate Core: iob_and

The simplest core description for Py2HWSW is as follows:

```
# SPDX-FileCopyrightText: 2026 IObundle
#
# SPDX-License-Identifier: GPL-3.0-only


def setup(py_params_dict):
    attributes_dict = {
        "generate_hw": True,
        "confs": [
            {
                "name": "general",
                "descr": "General group of confs",
                "confs": [
                    {
                        "name": "W",
                        "type": "P",
                        "val": "21",
                        "min": "1",
                        "max": "32",
                        "descr": "IO width",
                    },
                ],
            },
        ],
        "ports": [
            {
                "name": "a_i",
                "descr": "Input port",
                "signals": [
                    {"name": "a_i", "width": "W"},
                ],
            },
            {
                "name": "b_i",
                "descr": "Input port",
                "signals": [
                    {"name": "b_i", "width": "W"},
                ],
            },
            {
                "name": "y_o",
                "descr": "Output port",
                "signals": [
                    {"name": "y_o", "width": "W"},
                ],
            },
        ],
        "sw_modules": [
            {
                "core_name": "iob_coverage_analyze",
                "instance_name": "iob_coverage_analyze_inst",
```

```
            },
        ],
        "snippets": [{"verilog_code": "    assign y_o = a_i & b_i;"}],
    }

    return attributes_dict
```

View Source

A set of basic cores to showcase the various Py2HWSW features can be found in the basic_tests directory.

2.3 Setup And Build

To checkout the source and setup the example iob_and core:

```
$ git clone --recursive git@github.com:IObundle/py2hwsw.git
$ cd py2hwsw/
$ nix-shell py2hwsw/lib/ # Optional step to install environment with
   necessary dependencies
$ py2hwsw iob_and setup --no_verilog_lint
```

To do a clean setup:

```
$ py2hwsw iob_and clean
$ py2hwsw iob_and setup --no_verilog_lint
```

The setup process will generate a build directory containing the core's verilog sources and build files. By default, the build directory is '../[core_name]_V[core_version]'.

To build and run the core in simulation:

```
$ make -C ../iob_and_V* sim-run
```

2.4 Installation

Py2HWSW uses a Nix-shell environment to handle dependencies. The full list of dependencies is available as Nix packages in the `default.nix` file, which can be found at https://github.com/IObundle/py2hwsw/blob/main/py2hwsw/lib/default.nix.

The recommended way to install Py2HWSW is by using Nix-shell. Most Makefiles in IObundle projects call Nix-shell by default, so it is expected that a user will install Py2HWSW via Nix-shell. To do this, simply install Nix by following the instructions at https://nixos.org/download.html#nix-install-linux. Then, navigate to a directory that contains the Py2HWSW `default.nix` file and run `nix-shell`. Py2HWSW will self-install, and all dependencies will be installed automatically.

Alternatively, it is possible to install Py2HWSW manually by removing the Nix-shell commands from the Makefiles and installing the dependencies manually. After doing so, the user can call Py2HWSW by adding the `py2hwsw` file from the `bin/` folder to the `PATH` environment variable. The `py2hwsw` file can be found at https://github.com/IObundle/py2hwsw/blob/main/bin/py2hwsw.

Another option is to install Py2HWSW using pip with the following command:


```
pip install -e path/to/py2hwsw_directory
```

However, please note that this method is not officially supported, and dependencies will still need to be handled manually or by using Nix.

Py2HWSW is primarily maintained and tested on Linux, but it should also work on macOS and Windows Subsystem for Linux (WSL) since Nix is supported on these platforms.

2.5 Basic Usage

To use Py2HWSW, you can run the following command:

```
nix-shell --run "py2hwsw $(CORE) setup --build_dir '$(BUILD_DIR)' --
    py_params 'param1=param1_val:param2=param2_val"
```

This command sets up a core using Py2HWSW, where (CORE) is the name of the core, (BUILD DIR) is the directory where the build files will be generated, and (param1=param1_val:param2=param2_val) are optional Python parameters that can be used to customize the core.

The –build_dir option allows you to specify the location of the generated build directory. If not specified, the build directory will be generated in the parent directory of where the Py2HWSW command is called.

You can also use the –help option to list all available options and a brief description of each:

```
py2hwsw --help
```

To create a new core, you will need to create a setup directory with the same name as the core. This directory should contain at least one file with the same name as the core, either with a .py or .json extension, that describes the core using attributes of the core dictionary. The setup directory may also contain other files and folders following a standard hierarchy, which is described in more detail in other sections of this document.

For examples of simple cores, you can refer to the basic_tests folder in the Py2HWSW library: https://github.com/IObundle/py2hwsw/tree/main/py2hwsw/lib/hardware/basic_tests. For creating System On Chips, you can use the iob-soc repository as a template: https://github.com/IObundle/iob-soc/tree/main.

Some key concepts to understand when using Py2HWSW include:

- Setup directory: The folder that contains the core description and base files, templates, scripts, and sources.
- Build directory: The folder generated by the Py2HWSW setup process, which contains a standard file hierarchy and all the necessary makefiles to build and run the core on various simulators, FPGA, ASIC tools, and linters.
- Core: An IP core that contains Verilog sources, documentation, scripts, high-level attributes, and possibly software.
- Module: Sometimes used as an alternative to core, but it is recommended to use the term "core" instead. May also refer to Verilog modules and Python modules.

Py2HWSW can be used to create a wide variety of cores, from simple to complex. One of the main advantages of using Py2HWSW is that it generates readable Verilog code and all the necessary makefiles to run the core on various flows, making it a powerful tool for hardware design and development.

2.6 Universal Testbench

Py2HWSW supports a *Universal Test Bench*.

To use the *Universal Test Bench*, the core needs to provide the following files:

- iob_v_tb.vh
- iob_uut.v
- iob_core_tb.c
- sw_build.mk

Create the **iob_v_tb.vh** testbench header source and define the **IOB_CSRS_ADDR_W** macro to specify the address width of the simulation wrapper's CSRs bus (the width must be large enough to address all CSRs from all verification instruments). For example, the iob_uart core's simulation wrapper only uses one verification instrument (the iob_uart core itself). Therefore, the testbench should define the **IOB_CSRS_ADDR_W** macro to have the same width as the iob_uart core's CSRs bus. The iob_uart core's CSRs header files are also included because we can obtain the CSRs bus width from the auto-generated macro *IOB_UART_CSRS_ADDR_W*

```
// SPDX-FileCopyrightText: 2026 IObundle
//
// SPDX-License-Identifier: CERN-OHL-S-2.0

`include "iob_uart_csrs_conf.vh"
`define IOB_CSRS_ADDR_W `IOB_UART_CSRS_ADDR_W
```

View Source

Create **iob_uut.v** simulation wrapper and instantiate the verification instruments. For example, the iob_uart core is also used as a verification instrument to test itself. It is instantiated in the uart's iob_uut.v file, and its RS232 ports are connected in loopback. The iob_uut.v file is generated from the iob_uart_sim.py core's attributes (using the Py2HWSW attribute: "name": "iob_uut").

```
# SPDX-FileCopyrightText: 2026 IObundle
#
# SPDX-License-Identifier: GPL-3.0-only


def setup(py_params_dict):
    params = {
        # Type of interface for CSR bus
        "csr_if": "iob",
    }

    # Update params with values from py_params_dict
    for param in py_params_dict:
        if param in params:
            params[param] = py_params_dict[param]

    attributes_dict = {
        "name": "iob_uut",
        "generate_hw": True,
        "confs": [
```

```
            {
                "name": "DATA_W",
                "descr": "Data bus width",
                "type": "P",
                "val": "32",
            },
        ],
    }
    #
    # Ports
    #
    attributes_dict["ports"] = [
        {
            "name": "clk_en_rst_s",
            "descr": "Clock, clock enable and reset",
            "signals": {
                "type": "iob_clk",
            },
        },
        {
            "name": "uart_s",
            "descr": "Testbench uart csrs interface",
            "signals": {
                "type": "iob",
                "ADDR_W": 4,
            },
        },
    ]
    #
    # Wires
    #
    attributes_dict["wires"] = [
        {
            "name": "rs232_loopback",
            "descr": "Uart loopback wires",
            "signals": {
                "type": "rs232",
            },
        },
        {
            "name": "uart_cbus",
            "descr": "Testbench uart csrs bus",
            "signals": {
                "type": params["csr_if"],
                "prefix": "internal_",
                "ADDR_W": 4,
            },
        },
    ]
    #
    # Blocks
    #

    converter_connect = {
        "s_s": "uart_s",
```

```
        "m_m": "uart_cbus",
    }
    if params["csr_if"] != "iob":
        converter_connect["clk_en_rst_s"] = "clk_en_rst_s"
    attributes_dict["subblocks"] = [
        {
            "core_name": "iob_uart",
            "instance_name": "uart_inst",
            "instance_description": f"Unit Under Test (UUT) UART instance
               with '{params['csr_if']}' interface.",
            "csr_if": params["csr_if"],
            "connect": {
                "clk_en_rst_s": "clk_en_rst_s",
                "csrs_cbus_s": "uart_cbus",
                "rs232_m": "rs232_loopback",
            },
        },
        {
            "core_name": "iob_universal_converter",
            "instance_name": "iob_universal_converter",
            "instance_description": "Convert IOb port from testbench into
               correct interface for UART CSRs bus",
            "subordinate_if": "iob",
            "manager_if": params["csr_if"],
            "parameters": {
                "ADDR_W": 4,
                "DATA_W": "DATA_W",
            },
            "connect": converter_connect,
        },
    ]
    #
    # Snippets
    #
    attributes_dict["snippets"] = []
    snippet_code = """
   assign rs232_rxd = rs232_txd;
   assign rs232_cts = rs232_rts;
"""
    attributes_dict["snippets"] += [
        {"verilog_code": snippet_code},
    ]

    return attributes_dict
```

View Source

Create the **iob_core_tb.c** source to drive the verification instruments (instantiated in the simulation wrapper). For example, the iob_uart core's testbench drives this core, writing data to it, and reading back the data received from the loopback.

```
/*
 * SPDX-FileCopyrightText: 2026 IObundle
 *
 * SPDX-License-Identifier: GPL-3.0-only
```



```
 */

#include "iob_uart_csrs.h"
#include <stdio.h>

#define FREQ 100000000
#define BAUD 3000000

int iob_core_tb() {

  int failed = 0;

  // print welcome message
  printf("IOB UART testbench\n");

  // print the reset message
  printf("Reset complete\n");

  // hold soft reset low
  iob_uart_csrs_set_softreset(0);

  // print the soft reset message
  printf("Soft reset set to 0\n");

  // disable TX and RX

  iob_uart_csrs_set_txen(0);
  iob_uart_csrs_set_rxen(0);

  // set the divisor

  int div = FREQ / BAUD;
  iob_uart_csrs_set_div(div);

  // print the baud rate
  printf("Baud rate set to %d\n", BAUD);

  // assert tx and rx not ready
  uint8_t tx_ready = iob_uart_csrs_get_txready();
  if (tx_ready != 0) {
    printf("Error: TX ready initially\n");
    failed = 1;
  }

  uint8_t rx_ready = iob_uart_csrs_get_rxready();
  if (rx_ready != 0) {
    printf("Error: RX ready initially");
    failed = 1;
  }

  printf("TX and RX ready deasserted\n");

  // pulse soft reset
  iob_uart_csrs_set_softreset(1);
  iob_uart_csrs_set_softreset(0);
```

```
60
61    // enable RX
62    iob_uart_csrs_set_rxen(1);
63
64    unsigned int version;
65    int i;
66    // read version 20 times to burn time
67    for (i = 0; i < 20; i++) {
68      version = iob_uart_csrs_get_version();
69    }
70    printf("Version is %d\n", version);
71
72    // enable TX
73    iob_uart_csrs_set_txen(1);
74
75    printf("TX and RX enabled\n");
76
77    // data send/receive loop
78    printf("Starting data send/receive loop\n");
79    for (i = 0; i < 4; i++) {
80      // wait for tx ready
81      while (!iob_uart_csrs_get_txready())
82        ;
83
84      // write word to send
85      iob_uart_csrs_set_txdata(i);
86      // wait for rx ready
87      while (!iob_uart_csrs_get_rxready())
88        ;
89
90      // read received word
91      uint8_t rx_data = iob_uart_csrs_get_rxdata();
92
93      // check if received word is the same as sent word
94      if (rx_data != i) {
95        // signal error printing expected and received word
96        printf("Error: expected %d, received %d\n", i, rx_data);
97        failed += 1;
98      }
99    }
100   printf("Data send/receive loop complete\n");
101   return failed;
102 }
```

View Source

Create the **sw_build.mk** makefile segment and add the ‘tb‘ target to the ‘UTARGETS‘ list. Adding this target will cause the testbench software to be built. This testbench software will run on the host machine in parallel to the simulation. Also add the verification instrument’s CSRs sources to the ‘CSRS‘ list. Also update the ‘TB_INCLUDES‘ list as needed.

```
1 # SPDX-FileCopyrightText: 2026 IObundle
2 #
3 # SPDX-License-Identifier: GPL-3.0-only
4
```



```
5 UTARGETS=tb
```

View Source

3 How It Works

This section gives a detailed description of the Py2HWSW framework.

3.1 Overview

The Py2HWSW framework is organized into a repository with several key folders and scripts. The repository contains the main Py2HWSW module, as well as a library of cores and peripherals. The framework uses a combination of Python scripts and Makefiles to automate the generation of build directories for hardware components.

The setup process in Py2HWSW begins with the user providing a description of the core, which can be in the form of a Python script or a JSON file, in a setup directory. This description is then used to trigger the setup process, which involves gathering all dependency cores and generating the necessary Verilog code. The setup process creates a build directory, where all the generated Verilog modules are stored, correctly connected and structured based on the user's description. The build directory is independent of Py2HWSW and can be used on any machine with the necessary toolchain.

The build process is a separate step that takes the generated build directory as input and uses the Makefiles to run the toolchain for a specific flow, such as simulation or FPGA synthesis. For example, in the case of FPGA synthesis, the build process takes the generated Verilog sources as input, generates a bitstream, uploads it to the FPGA, and runs the design. In the case of simulation, the build process takes the Verilog sources and generates a simulator executable (for Verilator) or runs the simulation directly. The build process can be run on any machine with the necessary toolchain, without requiring Py2HWSW to be installed.

The main launch script, py2hwsw.py, serves as the entry point for the framework, and is responsible for orchestrating the setup process. The script takes care of setting up the build environment, generating Verilog code, and creating the build directory. Once the build directory is generated, the user can run the build process independently of Py2HWSW, using the Makefiles to automate the simulation, synthesis, and compilation of the hardware components.

3.2 Technical Details

From the user's perspective, interacting with Py2HWSW is straightforward and intuitive. Users describe cores using dictionaries, lists, and strings, which are then converted internally into object representations of the correct class. The main attributes of Py2HWSW, such as ports, wires, and configuration, each have their own class, organizing the properties of each attribute. These attributes are described by a dictionary, where each key is a property, and are converted to the corresponding property of the class for the internal object representation when calling the Py2HWSW process.

As described in the "Standard Interfaces" section, users only need to interact with Py2HWSW using standard interfaces based on dictionaries, lists, and strings. Internally, Py2HWSW converts these inputs into object representations, but these are usually only modified by developers. A typical user does not need to understand

the inner workings of Py2HWSW, making it easy to use and focus on designing and developing hardware components.

The iob_core.py class is the central component of Py2HWSW, aggregating all the properties of an IP core. Its constructor is responsible for the setup process of the core, which involves converting and initializing the attributes of the core, setting up submodules (each one a new iob_core instance), setting up superblocks (only if the current core is the top module or another superblock), and generating the sources for the current core in the build directory. If the current core is the top module, the setup process terminates by formatting and linting the code, as well as cleaning up temporary files from the build directory.

In terms of dependencies, Py2HWSW itself has a minimal set of requirements, including Python and certain Python libraries, as well as optional dependencies such as formatters and linters like Black, Verible, and Verilator. These formatters can be skipped if the user chooses not to use formatting and linting during setup. The generated build directory, on the other hand, may have additional dependencies specific to the build process, such as Makefiles, Verilog compilers and simulators. However, these dependencies are independent of Py2HWSW and are only required for the build process. Makefiles are not a required dependency of Py2HWSW itself, but can be useful for automating the setup process and integrating Py2HWSW into a larger project workflow.

3.3 Setup Flow Chart

Figure 1 presents a high-level flow chart of the Py2HWSW setup procedure.

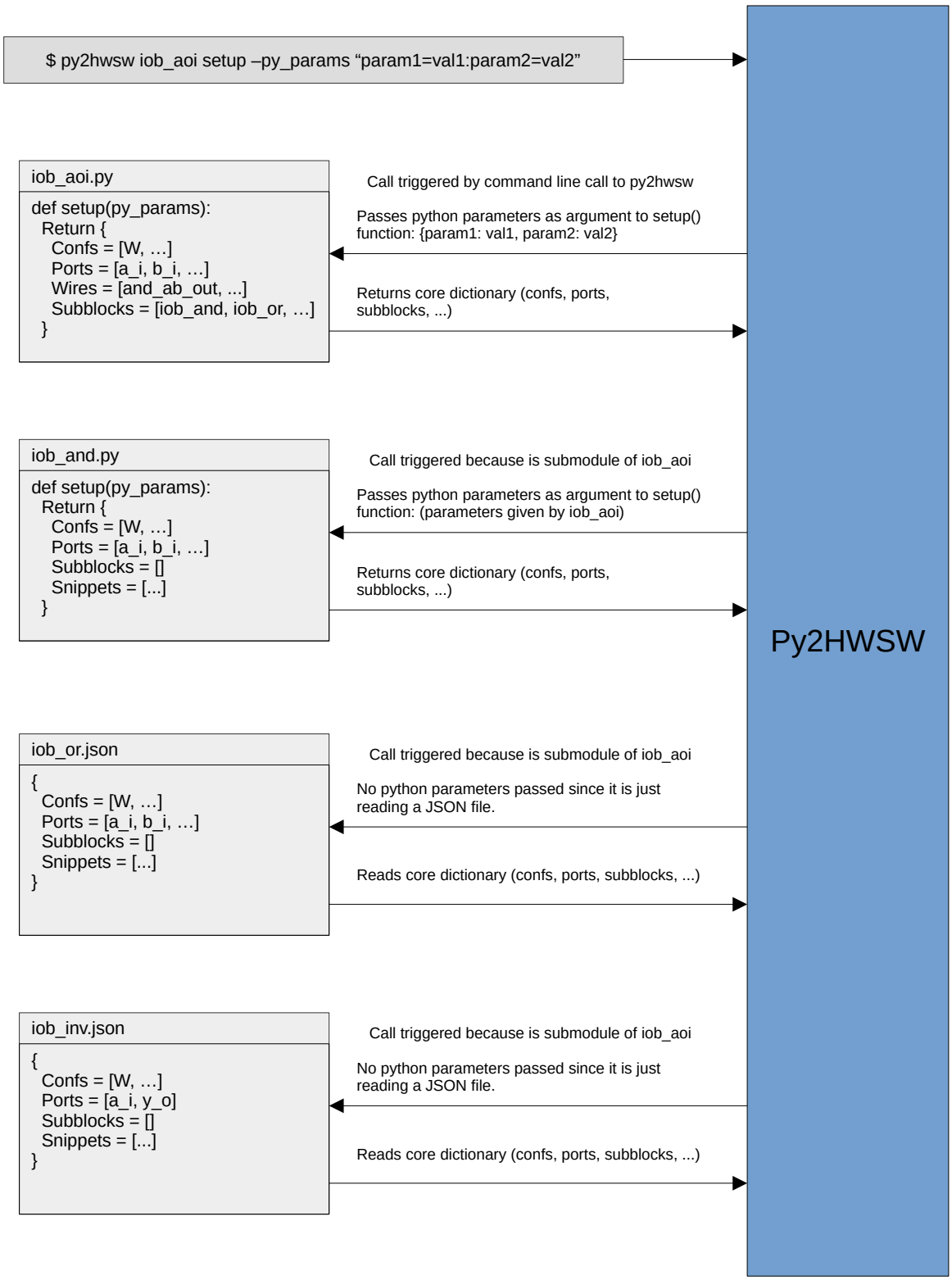

Figure 1: High-Level Flow Chart of Py2HWSW Setup Procedure

3.4 Standard Interfaces

The Py2HWSW framework provides the following two standard interfaces:

1. **Python Parameters**: Core "setup" function receives information from Py2HWSW via a dictionary in its first argument, referred to as *Python Parameters*.
2. **Core Dictionary**: Core "setup" function returns a core description dictionary to Py2HWSW, referred to as *Core Dictionary*.

The core's "setup" function is the python function defined by the user in the <core_name>.py file.

If the core is described by a JSON file, then the *Python Parameters* interface is not available. The JSON file gives a dictionary to Py2HWSW, similar to the python dictionary of the "setup" function. This allows the user to use external tools to generate cores in JSON format.

The *Python Parameters* received by the core's "setup" function is a dictionary containing both parameters passed by its issuer and standard parameters passed by Py2HWSW. Each key, value pair in the dictionary is a *Python Parameter*. The value of the python parameter may be of any data type.

Name	Data Type	Description
core_name	<class 'str'>	Name of current core (determined by the core's file name).
build_dir	<class 'str'>	Build directory of this core. Usually defined by '–build-dir' flag or issuer.
py2hwsw_target	<class 'str'>	The reason why py2hwsw is invoked. Usually 'setup' meaning the Py2HWSW is calling the core's script to obtain information on how to generate the core. May also be other targets like 'clean', 'print_attributes', or 'deliver'. These are usually to obtain information about the core for various purposes, but not to generate the build directory.
issuer	<class 'dict'>	Core dictionary with attributes of the issuer core (if any). Allows subblocks to obtain information about their issuer core.
py2hwsw_version	<class 'str'>	Version of Py2HWSW.

Table 1: Standard *Python Parameters* passed by Py2HWSW to every core's "setup" function.

The standard python parameters passed by Py2HWSW are listed in Table 1.

The python parameters supported by each core is available in the respective core's user guide, as long as they have the *Python Parameters* attribute defined. Instructions on how to build a core's user guide can be found in Section 4.8.

Name	Data Type	Description
original_name	<class 'str'>	Original name of the module. Should match name of module's *.py/*.json file. (The module name commonly used in the files of the setup dir.)
name	<class 'str'>	Name of the generated module.
description	<class 'str'>	Description of the module
reset_polarity	<class 'str'>	Global reset polarity of the module. Can be 'positive' or 'negative'. (Will override all subblocks' reset polarities).
confs	<class 'list'>	List of module macros and Verilog (false-)parameters.
ports	<class 'list'>	List of module ports.
wires	<class 'list'>	List of module wires.
snippets	<class 'list'>	List of core Verilog snippets.
comb	<class 'iob_comb.iob_comb'>	Verilog combinatory circuit.
fsm	<class 'iob_fsm.iob_fsm'>	Verilog finite state machine.
subblocks	<class 'list'>	List of instances of other cores inside this core.
superblocks	<class 'list'>	List of wrappers for this core. Will only be setup if this core is a top module, or a wrapper of the top module.
sw_modules	<class 'list'>	List of software modules required by this core.
instance_name	<class 'str'>	Name of the instance
instance_description	<class 'str'>	Description of the instance
portmap_connections	<class 'list'>	Instance portmap connections
parameters	typing.Dict	Verilog parameter values
instantiate	<class 'bool'>	Select if should intantiate the module inside another Verilog module.
build_dir	<class 'str'>	Path to folder of build directory to be generated for this project.
version	<class 'str'>	Core version in SemVer format (major.minor.patch). By default is the same as Py2HWSW version.
previous_version	<class 'str'>	Core previous version.
setup_dir	<class 'str'>	Path to root setup folder of the core.
use_netlist	<class 'bool'>	Copy '<SETUP_DIR>/CORE.v' netlist instead of '<SETUP_DIR>/hardware/src/*'
is_system	<class 'bool'>	Sets 'IS_FPGA=1' in config_build.mk
board_list	<class 'list'>	List of FPGAs supported by this core. A standard folder will be created for each board in this list.
dest_dir	<class 'str'>	Relative path inside build directory to copy sources of this core. Will only sources from 'hardware/src/*'
ignore_snippets	<class 'list'>	List of '.vs' file includes in verilog to ignore.
generate_hw	<class 'bool'>	Select if should try to generate '<corename>.v' from py2hwsw dictionary. Otherwise, only generate '.vs' files.
parent	<class 'dict'>	Select parent of this core (if any). If parent is set, that core will be used as a base for the current one. Any attributes of the current core will override/add to those of the parent.

is_top_module	<class 'bool'>	Selects if core is top module. Auto-filled. DO NOT CHANGE.
is_superblock	<class 'bool'>	Selects if core is superblock of another. Auto-filled. DO NOT CHANGE.
is_tester	<class 'bool'>	Generates makefiles and dependencies to run this core as if it was the top module. Used for testers (superblocks of top module).
python_parameters	<class 'list'>	List of core Python Parameters. Used for documentation.
license	<class 'iob_license.iob_license'>	License for the core.
doc_conf	<class 'str'>	CSR Configuration to use
title	<class 'str'>	Tile of this core. Used for documentation.

Table 2: Supported Py2HWSW attributes in the **Core Dictionary**. The *Data Type* column specifies the type of internal object that the Py2HWSW will convert the attribute's value to (usually the user inputs a string, list, or dictionary value and then py2 converts it to an internal object).

The list of attributes supported by the Py2HWSW framework is given in Table 2. If a core provides a dictionary with keys not listed in Table 2, then the Py2HWSW framework will raise an error. Each key, value pair in the dictionary is a *Core Attribute*. The data type of the core attribute may be of any data type, but are usually a string, list, or dictionary. If the data type is a string, it may also represent an object using Py2HWSW's *Short Notation*.

3.5 Block hierarchy

Figure 2 presents an example block hierarchy for a Py2HWSW project. Superblocks are only used if they are superblocks of the project's top module or of one of its wrappers.

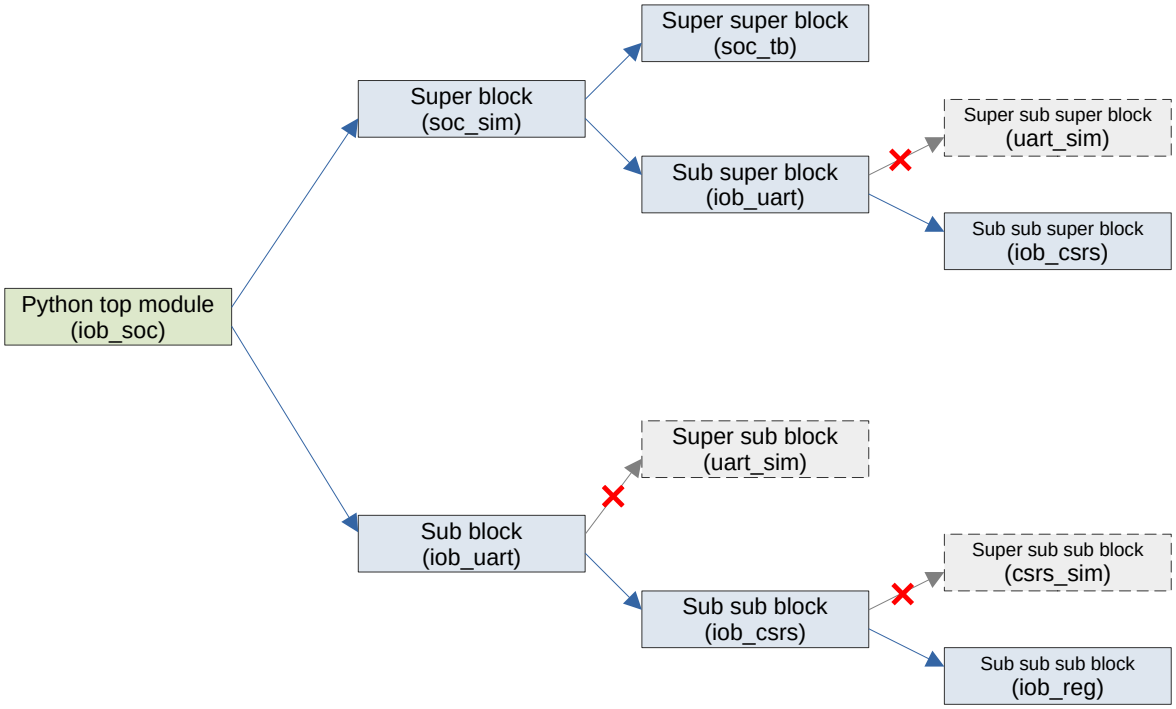

Figure 2: Block Hierarchy of a Py2HWSW Project

3.6 Main launch script: py2hwsw.py

The main launch script for the Py2HWSW program is the ‘py2hwsw.py‘ script.

The following code snippet from that script processes the command line arguments and launches the program for the specified "target".

```
    iob_core.global_build_dir = args.build_dir
    iob_core.global_project_root = args.project_root
    iob_core.global_project_vformat = args.verilog_format
    iob_core.global_project_vlint = args.verilog_lint
    iob_core.global_clang_format_rules_filepath = args.clang_rules
    iob_base.debug_level = args.debug_level

    if args.py2hwsw_docs:
        iob_core.setup_py2_docs(PY2HWSW_VERSION)
        exit(0)
    elif args.print_py2hwsw_attributes:
        iob_core.print_py2hwsw_attributes()
        exit(0)
    elif args.print_lib_cores:
        iob_core.print_lib_cores()
```

```
        exit(0)
    elif args.browse_lib:
        iob_core.browse_lib()
        exit(0)

    # Browse/Copy/Manage py2hwsw files
    # https://github.com/IObundle/iob-soc/pull/975#discussion_r1843025005
    for dir in dirs:
        py2hwsw_dir = os.path.join(
            os.path.dirname(os.path.realpath(__file__)), "..", dirs[dir]
        )
        if args.__dict__[f"{dir}_ls"]:
            list_dir(os.path.join(py2hwsw_dir, args.__dict__[f"{dir}_ls"]))
            exit(0)
        if args.__dict__[f"{dir}_cp"]:
            copy_dir(
                os.path.join(py2hwsw_dir, args.__dict__[f"{dir}_cp"][0]),
                args.__dict__[f"{dir}_cp"][1],
            )
            exit(0)
        if args.__dict__[f"{dir}_cat"]:
            cat_file(os.path.join(py2hwsw_dir, args.__dict__[f"{dir}_cat"]))
            exit(0)

    if not args.core_name:
        parser.print_usage(sys.stderr)
        exit(1)

    py_params = {}
    if args.py_params:
        for param in args.py_params.split(":"):
            k, v = param.split("=")
            py_params[k] = v

    if args.target == "setup":
        instance = iob_core.get_core_obj(args.core_name, **py_params)
        instance.generate_build_dir()
    elif args.target == "clean":
        iob_core.clean_build_dir(args.core_name)
    elif args.target == "print_build_dir":
        iob_core.print_build_dir(args.core_name, **py_params)
    elif args.target == "print_core_name":
        iob_core.print_core_name(args.core_name, **py_params)
    elif args.target == "print_core_version":
        iob_core.print_core_version(args.core_name, **py_params)
    elif args.target == "print_core_dict":
        iob_core.print_core_dict(args.core_name, **py_params)
    elif args.target == "deliver":
        iob_core.deliver_core(args.core_name, **py_params)
    elif args.target == "export_fusesoc":
        iob_core.export_fusesoc(args.core_name, **py_params)
```

View Source

3.7 Simulate with Verilator

With mandatory structured IOs, the testbench behaves like a processor reading and writing to its CSR. A universal Verilator testbench has been developed for an IP with a structured IOb native interface (bridges to standard AXI-Lite, APB or Wishbone are supplied). The testbench is a C++ program which provides hardware reset and CSR read and write functions.

3.7.1 IP core simulation

The IP cores using this testbench must provide a C function called `iob_core_tb()`, the IP core's specific test. They also must provide a C header called `iob_vlt_tb.h` that defines the Device Under Test (DUT) as a Verilator type called `dut_t`. With knowledge of the DUT and its test, the universal Verilator testbench will exercise any IP core. Interestingly, `iob_core_tb()` also runs, without modifications, on a RISC-V processor with the IP as a submodule, for example, for FPGA testing or emulation.

The `iob_uart` core is used as an example, located in the `py2hwsw/lib/peripherals/iob_uart` directory.

```
$ git clone --recursive git@github.com:IObundle/py2hwsw.git
$ cd py2hwsw/lib
$ make sim-run CORE=iob_uart SIMULATOR=verilator
```

The `make sim-run` command will run core setup, creating the build directory at `../../../iob_uart_V0.1`. The Verilator simulator will be run in the build directory. The testbench will be compiled and run, and the output will be displayed on the console.

3.7.2 Subsystem simulation

To illustrate system test capabilities with the universal Verilator testbench, the `iob_system` subsystem core is used as an example, located in the `py2hwsw/lib/iob_system` directory.

```
$ git clone --recursive git@github.com:IObundle/py2hwsw.git
$ cd py2hwsw/lib
$ make sim-run CORE=iob_system SIMULATOR=verilator
```

In this case the `iob_core_tb()` function is running on the desktop, emulating a system tester. The console output comes from the system itself running its embedded test, a more elaborated form of a hello world program.

3.8 Deliver an IP core

From the build directory, we select the essential files to create a tarball, all containing a Makefile-driven environment for the user who, in this way, will not need any ancillary tools beyond the standard EDA tools.

```
$ git clone --recursive git@github.com:IObundle/py2hwsw.git
$ cd py2hwsw/
$ nix-shell py2hwsw/lib/ # Optional step to install environment with
    necessary dependencies
$ py2hwsw iob_uart setup --no_verilog_lint
$ py2hwsw iob_uart deliver
```

The tarball will be created in the `../iob_uart_V0.1` directory, which is also the home of the default build directory.

4 Py2HWSW Classes

4.1 Main class for core representation: iob_core.py

The iob_core class is a central component of the Py2HWSW framework, responsible for representing and managing IP cores. The class provides a structured way to describe and generate IP cores, making it easier to create and integrate complex digital designs. At the heart of the iob_core class is its constructor, which plays a crucial role in setting up and initializing the core.

The constructor of the iob_core class is responsible for converting and initializing the attributes of the core, setting up subblocks, and generating the sources for the current core in the build directory. When the constructor is called, it distinguishes between two situations: when it is the top module, it creates a build directory for users with all the dependencies and project flows, and when it is a subblock, it provides all the information necessary for instantiation and integration. The constructor takes care of setting up the build environment, generating Verilog code, and creating the build directory, making it a key component of the Py2HWSW framework.

The iob_core constructor also handles the setup process for subblocks and superblocks. When a subblock is encountered, the constructor is called recursively to set up the subblock's attributes and generate its sources. Similarly, when a superblock is encountered, the constructor sets up the superblock's attributes and generates its sources, ensuring that the entire hierarchy of modules and subblocks is properly initialized and configured, as detailed in Section 3.5

Overall, the iob_core class and its constructor provide a powerful and flexible way to represent and manage IP cores, making it a fundamental component of the Py2HWSW framework.

It inherits attributes from its parent classes iob_module and iob_instance.

The get_core_obj function is used to generate an instance of a core based on a given core name and python parameters. This method will search for the corresponding Python or JSON file of the core, and generate a python object based on info stored in that file, and info passed via python parameters.

```
1     def get_core_obj(core_name, **kwargs):
2         """Generate an instance of a core based on given core_name and
              python parameters
3         This method will search for the .py and .json files of the core, and
               generate a
4         python object based on info stored in those files, and info passed
              via python
5         parameters.
6         Calling this method may also begin the setup process of the core,
              depending on
7         the value of the 'global_special_target' attribute.
8         """
9         core_dir, file_ext = find_module_setup_dir(core_name)
10
11        if file_ext == ".py":
12            import_python_module(
13                os.path.join(core_dir, f"{core_name}.py"),
```



```
            )
            core_module = sys.modules[core_name]
            issuer = kwargs.pop("issuer", None)
            top_module = __class__.global_top_module.original_name if
                __class__.global_top_module else core_name
            # Call ‘setup(<py_params_dict>)‘ function of ‘<core_name>.py‘ to
            # obtain the core’s py2hwsw dictionary.
            # Give it a dictionary with all arguments of this function,
                since the user
            # may want to use any of them to manipulate the core attributes.
            core_dict = core_module.setup(
                {
                    # "core_name": core_name,
                    "build_dir": __class__.global_build_dir,
                    "py2hwsw_target": __class__.global_special_target or "
                        setup",
                    "issuer": (issuer.attributes_dict if issuer else ""),
                    "py2hwsw_version": PY2HWSW_VERSION,
                    "top_module": top_module,
                    **kwargs,
                }
            )
            py2_core_dict = {
                "original_name": core_name,
                "name": core_name,
                "setup_dir": core_dir,
            }
            py2_core_dict.update(core_dict)
            instance = __class__.py2hw(
                py2_core_dict,
                issuer=issuer,
                # Note, any of the arguments below can have their values
                    overridden by
                # the py2_core_dict
                **kwargs,
            )
        elif file_ext == ".json":
            instance = __class__.read_py2hw_json(
                os.path.join(core_dir, f"{core_name}.json"),
                # Note, any of the arguments below can have their values
                    overridden by
                # the json data
                **kwargs,
            )
        return instance
```

View Source

4.2 Configuration class: iob_conf.py

The iob_conf class is used to represent a configuration option of the core. This class contains a set of attributes, each preceded by a comment describing the purpose of the attribute.

```python
class iob_conf_group:
    """Class to represent a group of configurations."""

    # Identifier name for the group of configurations.
    name: str = ""
    # Description of the configuration group.
    descr: str = "Default description"
    # List of configuration objects.
    confs: list = field(default_factory=list)
    # If enabled, configuration group will only appear in documentation. Not in the verilog code.
    doc_only: bool = False
    # If enabled, the documentation table for this group will be terminated by a TeX '\clearpage' command.
    doc_clearpage: bool = False
```

View Source

The iob_conf_group class is used to represent a group of configuration options. This class contains a set of attributes, each preceded by a comment describing the purpose of the attribute.


```python
class iob_conf_group:
    """Class to represent a group of configurations."""

    # Identifier name for the group of configurations.
    name: str = ""
    # Description of the configuration group.
    descr: str = "Default description"
    # List of configuration objects.
    confs: list = field(default_factory=list)
    # If enabled, configuration group will only appear in documentation. Not in the verilog code.
    doc_only: bool = False
    # If enabled, the documentation table for this group will be terminated by a TeX '\clearpage' command.
    doc_clearpage: bool = False
```


View Source

The py2hwsw tool uses methods from the config_gen.py script to generate the '*_conf.vh' file, which contains all the Verilog macros that must be held for every design instance of the core.

Each generated Verilog macro is based on the attributes from the corresponding instance of the 'iob_conf' class.

```python
def conf_vh(macros, top_module, out_dir):
    """Given a list with groups of macros, generate a '*_conf.vh' file with the Verilog macro definitions.
    :param macros: list with groups (iob_conf_group class) of macros (iob_conf class).
    :param top_module: top module name
    :param out_dir: output directory
    """
    os.makedirs(out_dir, exist_ok=True)
    file2create = open(f"{out_dir}/{top_module}_conf.vh", "w")
```



```
    core_prefix = f"{top_module}_".upper()

    # These ifndefs cause issues when this file is included in multiple files and it contains other ifdefs inside this block.
    # For example, assume this file is included in another one that does not have ‘MACRO1‘ defined. Now assume that inside this block there is an ‘ifdef MACRO1‘ block.
    # Since ‘MACRO1‘ is not defined in the file that is including this, the ‘ifdef MACRO1‘ wont be executed.
    # Now assume this file is included in another file that has ‘MACRO1‘ defined. Now, this block
    # wont execute because of the ‘ifndef‘ added here, therefore the ‘ifdef MACRO1‘ block will also not execute when it should have.
    # file2create.write(f"‘ifndef VH_{fname}_VH\n")
    # file2create.write(f"‘define VH_{fname}_VH\n\n")

    for group in macros:
        # If group has 'doc_only' attribute set to True, skip it
        if group.doc_only:
            continue

        file2create.write(f"// {group.name}: {group.descr}\n")

        # Sort macros macros by type, P first, M second, C third, D last
        sorted_macros = []
        for macro in group.confs:
            if macro.type == "P":
                sorted_macros.append(macro)
        for macro in group.confs:
            if macro.type == "M":
                sorted_macros.append(macro)
        for macro in group.confs:
            if macro.type == "C":
                sorted_macros.append(macro)
        for macro in group.confs:
            if macro.type == "D":
                sorted_macros.append(macro)

        prev_type = ""
        for macro in sorted_macros:
            macro_type = macro.type
            # If the type of the macro is different from the previous one, add a comment
            if macro_type != prev_type:
                if macro_type == "P":
                    file2create.write(
                        "// Core Configuration Parameters Default Values\n"
                    )
                elif macro_type == "M":
                    file2create.write("// Core Configuration Macros.\n")
                elif macro_type == "C":
                    file2create.write("// Core Constants. DO NOT CHANGE\n")
                elif macro_type == "D":
                    file2create.write("// Core Derived Parameters. DO NOT CHANGE\n")
```

```
                prev_type = macro_type

            # If macro has 'doc_only' attribute set to True, skip it
            if macro.doc_only:
                continue

            # Only insert macro if its is not a bool define, and if so only insert it if it is true
            if type(macro.val) is not bool:
                m_name = macro.name.upper()
                m_default_val = macro.val
                file2create.write(f"`define {core_prefix}{m_name} {m_default_val}\n")
            elif macro.val:
                m_name = macro.name.upper()
                file2create.write(f"`define {core_prefix}{m_name} 1\n")
```

View Source

The py2hwsw tool uses methods from the param_gen.py script to generate the Verilog parameters code that is automatically inserted in the core's Verilog module and instances.

Each generated Verilog parameter is based on the attributes from the corresponding instance of the 'iob_conf' class.

```
def generate_params(core):
    """Generate verilog code with verilog parameters of this module.
    returns: Generated verilog code
    """
    core_parameters = get_core_params(core.confs)

    if not core_parameters:
        return ""

    lines = []
    core_prefix = f"{core.name}_".upper()
    for idx, parameter in enumerate(core_parameters):
        # If parameter has 'doc_only' attribute set to True, skip it
        if parameter.doc_only:
            continue

        p_name = parameter.name.upper()
        p_comment = ""
        if parameter.type == "D":
            p_comment = "  // Don't change this parameter value!"
        lines.append(f"    parameter {p_name} = `{core_prefix}{p_name},{p_comment}\n")

    # Remove comma from last line
    if lines:
        lines[-1] = lines[-1].replace(",", "", 1)

    return "".join(lines)
```

View Source

4.3 Signal class: iob_signal.py

The iob_signal class is used to represent a signal for a hardware wire or port. This class contains a set of attributes, each preceded by a comment describing the purpose of the attribute.

View Source

The py2hwsw tool uses the ‘get_verilog_wire‘/‘get_verilog_port‘ methods from the ‘iob_signal‘ class to generate the Verilog code for the hardware wire/port based on the attributes from the corresponding instance of the ‘iob_signal‘ class.

```
    def get_verilog_wire(self):
        """Generate a verilog wire string from this signal"""
        if "'" in self.name or self.name.lower() == "z":
            return None
        wire_type = "reg" if self.isvar else "wire"
        width_str = "" if self.get_width_int() == 1 else f"[{self.width
            }-1:0] "
        return f"{wire_type} {width_str}{self.name};\n"
```

View Source

```
    def get_verilog_port(self, comma=True):
        """Generate a verilog port string from this signal"""
        if "'" in self.name or self.name.lower() == "z":
            return None
        self.assert_direction()
        comma_char = "," if comma else ""
        port_type = " reg" if self.isvar else ""
        width_str = "" if self.get_width_int() == 1 else f"[{self.width
            }-1:0] "
        return f"{self.direction}{port_type} {width_str}{self.name}{
            comma_char}\n"
```

View Source

4.4 Wire class: iob_wire.py

The iob_wire class is used to represent a group of hardware wires (signals) used to interconnect components automatically generated. This class contains a set of attributes, each preceded by a comment describing the purpose of the attribute.

```
class iob_wire:
    """Class to represent a wire in an iob module"""

    # Identifier name for the wire.
    name: str = ""
    # Name of the standard interface to auto-generate with ‘interfaces.py‘
        script.
    interface: interfaces._interface = None
    # Description of the wire.
    descr: str = "Default description"
```

```
    # List of signals belonging to this wire
    # (each signal represents a hardware Verilog wire).
    signals: List = field(default_factory=list)
```

View Source

The ‘signals‘ attribute stores a list of signal objects, represented by the ‘iob_signal‘ class (Section 4.3).

The py2hwsw tool uses the ‘generate_wires‘ method from the ‘wire_gen.py‘ script to generate the Verilog code for the wire based on the attributes from the corresponding instance of the ‘iob_wire‘ class.

```
def generate_wires(core):
    """Generate verilog code with wires of this module.
    returns: Generated verilog code
    """
    code = ""
    for wire in core.wires:

        signals_code = ""
        for signal in wire.signals:
            if isinstance(signal, iob_signal):
                if signal.get_verilog_wire():
                    signals_code += "    " + signal.get_verilog_wire()
        if signals_code:
            # Add description for the wire if it is not the default one
            if wire.descr != "" and wire.descr != "Default description":
                code += f"// {wire.descr}\n"
            code += signals_code

    return code
```

View Source

4.5 Port class: iob_port.py

The iob_port class is used to represent an interface for the core. An interface is a group of hardware ports (signals) that may be generic or follow a standard. Due to the similarities between a port and a wire, this class inherits the attributes from the ‘iob_wire‘ class (Section 4.4). Besides the inherited attributes, the class contains a set of new port specific attributes, each preceded by a comment describing the purpose of the attribute.

```
class iob_port(iob_wire):
    """Describes an IO port."""

    doc_only: bool = False
    # If enabled, the documentation table for this port will be terminated
       by a TeX '\clearpage' command.
    doc_clearpage: bool = False
```

View Source

Similar to the ‘iob_wire‘ class, the ‘signals‘ attribute stores a list of signal objects, represented by the ‘iob_signal‘ class (Section 4.3).

The py2hwsw tool uses the ‘generate_ports‘ method from the ‘io_gen.py‘ script to generate the Verilog code for the port based on the attributes from the corresponding instance of the ‘iob_port‘ class.

```
def generate_ports(core):
    """Generate verilog code with ports of this module.
    returns: Generated verilog code
    """
    lines = []
    for port_idx, port in enumerate(core.ports):
        # If port has 'doc_only' attribute set to True, skip it
        if port.doc_only:
            continue

        lines.append(f"    // {port.name}: {port.descr}\n")

        for signal_idx, signal in enumerate(port.signals):
            if isinstance(signal, iob_signal):
                if signal.get_verilog_port():
                    lines.append("    " + signal.get_verilog_port())

    # Remove comma from last port line
    if lines:
        i = -1
        while lines[i].startswith("`endif") or lines[i].startswith("    // "
            ):
            i -= 1
        lines[i] = lines[i].replace(",", "", 1)

    return "".join(lines)
```

View Source

4.6 Interface class: interfaces.py

The Py2HWSW tool uses the interfaces.py script to generate the signals of standard interfaces.

The standard interfaces currently supported by the interfaces.py script are listed below.

```
mem_if_details  = [
    {
        "name": "rom_2p",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "ROM 2 Port",
    },
    {
        "name": "rom_atdp",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "ROM Asynchronous True Dual Port",
    },
```

```
    {
        "name": "rom_sp",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "ROM Single Port",
    },
    {
        "name": "rom_tdp",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "ROM True Dual Port",
    },
    {
        "name": "ram_2p",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM 2 Port",
    },
    {
        "name": "ram_at2p",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM Asynchronous True 2 Port",
    },
    {
        "name": "ram_atdp",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM Asynchronous True Dual Port",
    },
    {
        "name": "ram_atdp_be",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM Asynchronous True Dual Port with Byte Enable",
    },
    {
        "name": "ram_sp",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM Single Port",
    },
    {
        "name": "ram_sp_be",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM Single Port with Byte Enable",
```



```
    },
    {
        "name": "ram_sp_se",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM Single Port with Single Enable",
    },
    {
        "name": "ram_t2p",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM True 2 Port",
    },
    {
        "name": "ram_t2p_be",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM True 2 Port with Byte Enable",
    },
    {
        "name": "ram_t2p_tiled",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM True 2 Port Tiled",
    },
    {
        "name": "ram_tdp",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM True Dual Port",
    },
    {
        "name": "ram_tdp_be",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM True Dual Port with Byte Enable",
    },
    {
        "name": "ram_tdp_be_xil",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "RAM True Dual Port with Byte Enable Xilinx",
    },
    {
        "name": "regfile_2p",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
```

```
        "full_name": "Register File 2 Port",
    },
    {
        "name": "regfile_at2p",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "Register File Asynchronous True 2 Port",
    },
    {
        "name": "regfile_sp",
        "vendor": "IObundle",
        "lib": "MEM",
        "version": "1.0",
        "full_name": "Register File Single Port",
    },
]
```

View Source

```
if_details  = [
    {
        "name": "iob_clk",
        "vendor": "IObundle",
        "lib": "CLK",
        "version": "1.0",
        "full_name": "Clock (configurable)",
    },
    {
        "name": "iob",
        "vendor": "IObundle",
        "lib": "IOb",
        "version": "1.0",
        "full_name": "IOb",
    },
    {
        "name": "axil_read",
        "vendor": "ARM",
        "lib": "AXI",
        "version": "4.0",
        "full_name": "AXI-Lite Read",
    },
    {
        "name": "axil_write",
        "vendor": "ARM",
        "lib": "AXI",
        "version": "4.0",
        "full_name": "AXI-Lite Write",
    },
    {
        "name": "axil",
        "vendor": "ARM",
        "lib": "AXI",
        "version": "4.0",
        "full_name": "AXI-Lite",
```



```
    },
    {
        "name": "axi_read",
        "vendor": "ARM",
        "lib": "AXI",
        "version": "4.0",
        "full_name": "AXI Read",
    },
    {
        "name": "axi_write",
        "vendor": "ARM",
        "lib": "AXI",
        "version": "4.0",
        "full_name": "AXI Write",
    },
    {
        "name": "axi",
        "vendor": "ARM",
        "lib": "AXI",
        "version": "4.0",
        "full_name": "AXI",
    },
    {
        "name": "apb",
        "vendor": "ARM",
        "lib": "APB",
        "version": "4.0",
        "full_name": "APB",
    },
    {
        "name": "ahb",
        "vendor": "ARM",
        "lib": "AHB",
        "version": "4.0",
        "full_name": "AHB",
    },
    {
        "name": "axis",
        "vendor": "ARM",
        "lib": "AXI",
        "version": "4.0",
        "full_name": "AXI Stream",
    },
    {
        "name": "rs232",
        "vendor": "Generic",
        "lib": "RS232",
        "version": "1.0",
        "full_name": "RS232",
    },
    {
        "name": "mii",
        "vendor": "Generic",
        "lib": "MII",
        "version": "1.0",
```

```
        "full_name": "Media Independent Interface",
    },
    {
        "name": "wb",
        "vendor": "OPENCORES",
        "lib": "Wishbone",
        "version": "B4",
        "full_name": "Wishbone",
    },
    {
        "name": "wb_full",
        "vendor": "OPENCORES",
        "lib": "Wishbone",
        "version": "B4",
        "full_name": "Wishbone Full",
    },
]
```

View Source

When a user specifies a standard interface for a port 4.5 or a wire 4.4, a new instance of the '_interface' subclass for that interface is created to represent it.

```
class _interface:
    """Class to represent an interface for generation"""

    # Interface direction. Examples: '' (unspecified), 'manager', '
       subordinate', ...
    if_direction: str = ""
    # Prefix for signals of the interface
    prefix: str = ""
    # Width multiplier. Used when concatenating multiple instances of the
       interface.
    mult: str | int = 1
    # Prefix for generated "Verilog Snippets" of this interface
    file_prefix: str = ""
    # Prefix for "Verilog snippets" of portmaps of this interface:
    portmap_port_prefix: str = ""
    # List of signals for this interface (Internal, used for generation)
    _signals: list = field(default_factory=list)
```

View Source

This class stores the interface properties which will then be used by interface specific functions to generate the signals. Each attribute of the interface class is preceded by a comment describing the purpose of the attribute.

Each interface supported by interfaces.py is implemented by its own subclass of '_interface', which defines the signals of that interface.

AXI Stream

The AXI Stream interface is implemented by the ‘AXIStreamInterface‘ class of the interfaces.py script.

```
class AXIStreamInterface(_interface):
    """Class to represent an AXI-Stream interface for generation"
       ""

    # Data width for the AXI-Stream interface
    data_w: int = 32
    # Signal to indicate if the interface has a last signal
    has_tlast: bool = False

    def __post_init__(self):
        super().__post_init__()
        self.__set_signals()

    def __set_signals(self):
        """Set signals for the AXI-Stream interface."""
        self._signals += [
            iob_signal(
                name="axis_tdata_o",
                width=self.data_w,
                descr="AXI-Stream data output.",
            ),
            iob_signal(
                name="axis_tvalid_o",
                descr="AXI-Stream valid output.",
            ),
            iob_signal(
                name="axis_tready_i",
                descr="AXI-Stream ready input.",
            ),
        ]

        if self.has_tlast:
            self._signals.append(
                iob_signal(
                    name="axis_tlast_o",
                    descr="AXI-Stream last signal output.",
                )
            )
```

View Source

It has the configurable width: DATA_W

For example, to add an AXI Stream port to a core, add the following python dictionary to the core's ports list:

```
{
    "name": "example_port_m",
    "descr": "AXI Stream manager port",
    "signals": {
        "type": "axis",
        "DATA_W": 32,
    },
},
```


AXI Lite

The AXI Lite interface is implemented by the ‘AXILiteInterface‘ class of the interfaces.py script.

```
class AXILiteInterface(_interface):
    """Class to represent an AXI-Lite interface for generation"""

    # Data width for the AXI-Lite interface
    data_w: int or str = 32
    strb_w: int or str = None
    # Address width for the AXI-Lite interface
    addr_w: int or str = 32
    # AXI-Lite parameters
    resp_w: int or str = 2
    prot_w: int or str = 3
    # Interfaces/Ports to include in the AXI-Lite interface
    has_read_if: bool = True
    has_write_if: bool = True
    has_prot: bool = False

    def __post_init__(self):
        super().__post_init__()
        if not self.strb_w:
            self.strb_w = f"{self.data_w}/8"
        self.__set_signals()

    def __set_signals(self):
        """Set signals for the AXI-Lite interface."""

        if self.has_read_if:
            self.__set_read_signals()
        if self.has_write_if:
            self.__set_write_signals()

    def __set_write_signals(self):
        """Set write signals for the AXI-Lite interface."""
        self._signals += [
            iob_signal(
                name="axil_awaddr_o",
                width=self.addr_w,
                descr="AXI-Lite address write channel byte
                   address.",
            ),
        ]
        if self.has_prot:
            self._signals.append(
                iob_signal(
```

```
                    name="axil_awprot_o",
                    width=self.prot_w,
                    descr="AXI-Lite address write channel
                       protection type.",
                )
            )
        self._signals += [
            iob_signal(
                name="axil_awvalid_o",
                descr="AXI-Lite address write channel valid.",
            ),
            iob_signal(
                name="axil_awready_i",
                descr="AXI-Lite address write channel ready.",
            ),
            iob_signal(
                name="axil_wdata_o",
                width=self.data_w,
                descr="AXI-Lite write channel data.",
            ),
            iob_signal(
                name="axil_wstrb_o",
                width=self.strb_w,
                descr="AXI-Lite write channel write strobe.",
            ),
            iob_signal(
                name="axil_wvalid_o",
                descr="AXI-Lite write channel valid.",
            ),
            iob_signal(
                name="axil_wready_i",
                descr="AXI-Lite write channel ready.",
            ),
            iob_signal(
                name="axil_bresp_i",
                width=self.resp_w,
                descr="AXI-Lite write response channel response."
                   ,
            ),
            iob_signal(
                name="axil_bvalid_i",
                descr="AXI-Lite write response channel valid.",
            ),
            iob_signal(
                name="axil_bready_o",
                descr="AXI-Lite write response channel ready.",
            ),
```



```
        ]

    def __set_read_signals(self):
        """Set read signals for the AXI-Lite interface."""
        self._signals += [
            iob_signal(
                name="axil_araddr_o",
                width=self.addr_w,
                descr="AXI-Lite address read channel byte address
                    .",
            ),
        ]
        if self.has_prot:
            self._signals.append(
                iob_signal(
                    name="axil_arprot_o",
                    width=self.prot_w,
                    descr="AXI-Lite address read channel
                        protection type.",
                )
            )
        self._signals += [
            iob_signal(
                name="axil_arvalid_o",
                descr="AXI-Lite address read channel valid.",
            ),
            iob_signal(
                name="axil_arready_i",
                descr="AXI-Lite address read channel ready.",
            ),
            iob_signal(
                name="axil_rdata_i",
                width=self.data_w,
                descr="AXI-Lite read channel data.",
            ),
            iob_signal(
                name="axil_rresp_i",
                width=self.resp_w,
                descr="AXI-Lite read channel response.",
            ),
            iob_signal(
                name="axil_rvalid_i",
                descr="AXI-Lite read channel valid.",
            ),
            iob_signal(
                name="axil_rready_o",
                descr="AXI-Lite read channel ready.",
```

```
133                 ),
134             ]
```

View Source

It has the configurable widths:

- ADDR_W
- DATA_W
- PROT_W
- RESP_W

The AXI Lite interface also supports the following 'params':

- prot: Include "prot" signal

For example, to add an AXI Lite port to a core, add the following python dictionary to the core's ports list:

```
{
    "name": "example_port_m",
    "descr": "AXI Lite manager port",
    "signals": {
        "type": "axil",
        "ADDR_W": 32,
        "DATA_W": 32,
        "PROT_W": 3,
        "RESP_W": 2,
    },
},
```


AXI

The AXI interface is implemented by the ‘AXIInterface‘ class of the interfaces.py script.

```
class AXIInterface(_interface):
    """Class to represent an AXI interface for generation"""

    # Data width for the AXI interface
    data_w: int or str = 32
    strb_w: int or str = None
    # Address width for the AXI interface
    addr_w: int or str = 32
    # AXI parameters
    id_w: int or str = 1
    size_w: int or str = 3
    burst_w: int or str = 2
    lock_w: int or str = 2
    cache_w: int or str = 4
    prot_w: int or str = 3
    qos_w: int or str = 4
    resp_w: int or str = 2
    len_w: int or str = 8
    # Interfaces/Ports to include in the AXI-Lite interface
    has_read_if: bool = True
    has_write_if: bool = True
    has_prot: bool = False

    def __post_init__(self):
        super().__post_init__()
        if not self.strb_w:
            self.strb_w = f"{self.data_w}/8"
        self.__set_signals()

    def __set_signals(self):
        """Set signals for the AXI interface."""

        if self.has_read_if:
            self.__set_read_signals()
        if self.has_write_if:
            self.__set_write_signals()

    def __set_write_signals(self):
        """Set write signals for the AXI interface."""

        # AXI-Lite write signals
        self._signals += [
            iob_signal(
                name="axi_awaddr_o",
```

```
                width=self.addr_w,
                descr="AXI address write channel byte address.",
            ),
        ]
        if self.has_prot:
            self._signals.append(
                iob_signal(
                    name="axi_awprot_o",
                    width=self.prot_w,
                    descr="AXI address write channel protection
                       type.",
                )
            )
        self._signals += [
            iob_signal(
                name="axi_awvalid_o",
                descr="AXI address write channel valid.",
            ),
            iob_signal(
                name="axi_awready_i",
                descr="AXI address write channel ready.",
            ),
            iob_signal(
                name="axi_wdata_o",
                width=self.data_w,
                descr="AXI write channel data.",
            ),
            iob_signal(
                name="axi_wstrb_o",
                width=self.strb_w,
                descr="AXI write channel write strobe.",
            ),
            iob_signal(
                name="axi_wvalid_o",
                descr="AXI write channel valid.",
            ),
            iob_signal(
                name="axi_wready_i",
                descr="AXI write channel ready.",
            ),
            iob_signal(
                name="axi_bresp_i",
                width=self.resp_w,
                descr="AXI write response channel response.",
            ),
            iob_signal(
                name="axi_bvalid_i",
```



```
                descr="AXI write response channel valid.",
            ),
            iob_signal(
                name="axi_bready_o",
                descr="AXI write response channel ready.",
            ),
        ]

        # AXI write channel signals
        self._signals += [
            iob_signal(
                name="axi_awid_o",
                width=self.id_w,
                descr="AXI address write channel ID.",
            ),
            iob_signal(
                name="axi_awlen_o",
                width=self.len_w,
                descr="AXI address write channel burst length.",
            ),
            iob_signal(
                name="axi_awsize_o",
                width=self.size_w,
                descr="AXI address write channel burst size.",
            ),
            iob_signal(
                name="axi_awburst_o",
                width=self.burst_w,
                descr="AXI address write channel burst type.",
            ),
            iob_signal(
                name="axi_awlock_o",
                width=self.lock_w,
                descr="AXI address write channel lock type.",
            ),
            iob_signal(
                name="axi_awcache_o",
                width=self.cache_w,
                descr="AXI address write channel memory type.",
            ),
            iob_signal(
                name="axi_awqos_o",
                width=self.qos_w,
                descr="AXI address write channel quality of
                   service.",
            ),
            iob_signal(
```

```
                name="axi_wlast_o",
                descr="AXI Write channel last word flag.",
            ),
            iob_signal(
                name="axi_bid_i",
                width=self.id_w,
                descr="AXI Write response channel ID.",
            ),
        ]

    def __set_read_signals(self):
        """Set read signals for the AXI interface."""

        # AXI-Lite read signals
        self._signals += [
            iob_signal(
                name="axi_araddr_o",
                width=self.addr_w,
                descr="AXI address read channel byte address.",
            ),
        ]
        if self.has_prot:
            self._signals.append(
                iob_signal(
                    name="axi_arprot_o",
                    width=self.prot_w,
                    descr="AXI address read channel protection
                       type.",
                )
            )
        self._signals += [
            iob_signal(
                name="axi_arvalid_o",
                descr="AXI address read channel valid.",
            ),
            iob_signal(
                name="axi_arready_i",
                descr="AXI address read channel ready.",
            ),
            iob_signal(
                name="axi_rdata_i",
                width=self.data_w,
                descr="AXI read channel data.",
            ),
            iob_signal(
                name="axi_rresp_i",
                width=self.resp_w,
```



```
                descr="AXI read channel response.",
            ),
            iob_signal(
                name="axi_rvalid_i",
                descr="AXI read channel valid.",
            ),
            iob_signal(
                name="axi_rready_o",
                descr="AXI read channel ready.",
            ),
        ]

        # AXI read channel signals
        self._signals += [
            iob_signal(
                name="axi_arid_o",
                width=self.id_w,
                descr="AXI address read channel ID.",
            ),
            iob_signal(
                name="axi_arlen_o",
                width=self.len_w,
                descr="AXI address read channel burst length.",
            ),
            iob_signal(
                name="axi_arsize_o",
                width=self.size_w,
                descr="AXI address read channel burst size.",
            ),
            iob_signal(
                name="axi_arburst_o",
                width=self.burst_w,
                descr="AXI address read channel burst type.",
            ),
            iob_signal(
                name="axi_arlock_o",
                width=self.lock_w,
                descr="AXI address read channel lock type.",
            ),
            iob_signal(
                name="axi_arcache_o",
                width=self.cache_w,
                descr="AXI address read channel memory type.",
            ),
            iob_signal(
                name="axi_arqos_o",
                width=self.qos_w,
```

```
                descr="AXI address read channel quality of
                    service.",
            ),
            iob_signal(
                name="axi_rid_i",
                width=self.id_w,
                descr="AXI Read channel ID.",
            ),
            iob_signal(
                name="axi_rlast_i",
                descr="AXI Read channel last word.",
            ),
        ]
```

View Source

The AXI interface extends configurable widths of the AXI Lite interface, with the following additions:

- ID_W
- SIZE_W
- BURST_W
- LOCK_W
- CACHE_W
- QOS_W
- LEN_W

The AXI interface supports the same 'params' as the AXI Lite interface.

For example, to add an AXI port to a core, add the following python dictionary to the core's ports list:

```
{
    "name": "example_port_m",
    "descr": "AXI manager port",
    "signals": {
        "type": "axi",
        "ADDR_W": 32,
        "DATA_W": 32,
        "PROT_W": 3,
        "RESP_W": 2,
        "ID_W": 4,
        "SIZE_W": 3,
        "BURST_W": 2,
```



```
            "LOCK_W": 2,
            "CACHE_W": 4,
            "QOS_W": 4,
            "LEN_W": 8,
        },
},
```

APB

The APB interface is implemented by the ‘APBInterface‘ class of the interfaces.py script.

```
class APBInterface(_interface):
    """Class to represent an APB interface for generation"""

    # Data width for the APB interface
    data_w: int or str = 32
    strb_w: int or str = None
    # Address width for the APB interface
    addr_w: int or str = 32

    def __post_init__(self):
        super().__post_init__()
        if not self.strb_w:
            self.strb_w = f"{self.data_w}/8"
        self.__set_signals()

    def __set_signals(self):
        """Set signals for the APB interface."""
        self._signals += [
            iob_signal(
                name="apb_addr_o",
                width=self.addr_w,
                descr="APB address output.",
            ),
            iob_signal(
                name="apb_sel_o",
                descr="APB subordinate select.",
            ),
            iob_signal(
                name="apb_enable_o",
                descr="APB enable. Indicates the number of clock
                    cycles of the transfer.",
            ),
            iob_signal(
                name="apb_write_o",
                descr="APB write. Indicates the direction of the
                    operation.",
            ),
            iob_signal(
                name="apb_wdata_o",
                width=self.data_w,
                descr="APB write data.",
            ),
            iob_signal(
                name="apb_wstrb_o",
```



```
                    width=self.strb_w,
                    descr="APB write strobe.",
                ),
                iob_signal(
                    name="apb_rdata_i",
                    width=self.data_w,
                    descr="APB read data.",
                ),
                iob_signal(
                    name="apb_ready_i",
                    descr="APB ready. Indicates the end of a transfer
                        .",
                ),
            ]
```

View Source

The APB interface has the configurable widths:

- ADDR_W
- DATA_W

For example, to add an APB port to a core, add the following python dictionary to the core's ports list:

```
{
    "name": "example_port_m",
    "descr": "APB manager port",
    "signals": {
        "type": "apb",
        "ADDR_W": 32,
        "DATA_W": 32,
    },
},
```

AHB

The AHB interface is implemented by the ‘AHBInterface‘ class of the interfaces.py script.

```
class AHBInterface(_interface):
    """Class to represent an AHB interface for generation"""

    # Data width for the AHB interface
    data_w: int or str = 32
    strb_w: int or str = None
    # Address width for the AHB interface
    addr_w: int or str = 32
    # AHB parameters
    prot_w: int or str = 4
    burst_w: int or str = 3
    trans_w: int or str = 2
    size_w: int or str = 3

    def __post_init__(self):
        super().__post_init__()
        if not self.strb_w:
            self.strb_w = f"{self.data_w}/8"
        self.__set_signals()

    def __set_signals(self):
        """Set signals for the AHB interface."""
        self._signals += [
            iob_signal(
                name="ahb_addr_o",
                width=self.addr_w,
                descr="AHB address output.",
            ),
            iob_signal(
                name="ahb_burst_o",
                width=self.burst_w,
                descr="AHB burst size.",
            ),
            iob_signal(
                name="ahb_mastlock_o",
                descr="AHB manager lock signal.",
            ),
            iob_signal(
                name="ahb_prot_o",
                width=self.prot_w,
                descr="AHB protection type.",
            ),
            iob_signal(
                name="ahb_size_o",
```



```
                width=self.size_w,  # Size is typically 3 bits in AHB
                descr="AHB transfer size.",
            ),
            iob_signal(
                name="ahb_trans_o",
                width=self.trans_w,
                descr="AHB transfer type.",
            ),
            iob_signal(
                name="ahb_wdata_o",
                width=self.data_w,
                descr="AHB write data.",
            ),
            iob_signal(
                name="ahb_wstrb_o",
                width=self.strb_w,
                descr="AHB write strobe.",
            ),
            iob_signal(
                name="ahb_write_o",
                descr="AHB write signal. Transfer direction: (1) Write; (0) Read.",
            ),
            iob_signal(
                name="ahb_rdata_i",
                width=self.data_w,
                descr="AHB read data.",
            ),
            iob_signal(
                name="ahb_readyout_i",
                descr="AHB ready output. Indicates transfer finished on the bus.",
            ),
            iob_signal(
                name="ahb_resp_i",
                descr="AHB response input. Transfer response: (0) Okay; (1) Error.",
            ),
            iob_signal(
                name="ahb_sel_o",
                descr="AHB subordinate select.",
            ),
        ]
```

View Source

The AHB interface has the configurable widths:

- ADDR_W
- DATA_W
- BURST_W
- PROT_W
- SIZE_W
- TRANS_W

For example, to add an AHB port to a core, add the following python dictionary to the core's ports list:

```
{
    "name": "example_port_m",
    "descr": "AHB manager port",
    "signals": {
        "type": "ahb",
        "ADDR_W": 32,
        "DATA_W": 32,
        "BURST_W": 3,
        "PROT_W": 4,
        "SIZE_W": 3,
        "TRANS_W": 2,
    },
},
```


Wishbone

The Wishbone interface is implemented by the ‘wishboneInterface‘ class of the interfaces.py script.

```
class wishboneInterface(_interface):
    """Class to represent a Wishbone interface for generation"""

    # Data width for the Wishbone interface
    data_w: int or str = 32
    strb_w: int or str = None
    # Address width for the Wishbone interface
    addr_w: int or str = 32
    # Sets if it is a full Wishbone interface
    is_full: bool = False

    def __post_init__(self):
        super().__post_init__()
        if not self.strb_w:
            self.strb_w = f"{self.data_w}/8"
        self.__set_signals()

    def __set_signals(self):
        """Set signals for the Wishbone interface."""
        self._signals += [
            iob_signal(
                name="wb_dat_i",
                width=self.data_w,
                descr="Data input.",
            ),
            iob_signal(
                name="wb_datout_o",
                width=self.data_w,
                descr="Data output.",
            ),
            iob_signal(
                name="wb_ack_i",
                descr="Acknowledge input. Indicates normal
                   termination of a bus cycle.",
            ),
            iob_signal(
                name="wb_adr_o",
                width=self.addr_w,
                descr="Address output. Passes binary address.",
            ),
            iob_signal(
                name="wb_cyc_o",
```

```
42                  descr="Cycle output. Indicates a valid bus cycle.
                        ",
43              ),
44              iob_signal(
45                  name="wb_sel_o",
46                  width=self.strb_w,
47                  descr="Select output. Indicates where valid data
                        is expected on the data bus.",
48              ),
49              iob_signal(
50                  name="wb_stb_o",
51                  descr="Strobe output. Indicates valid access.",
52              ),
53              iob_signal(
54                  name="wb_we_o",
55                  descr="Write enable. Indicates write access.",
56              ),
57          ]
58
59          if self.is_full:
60              self._signals += [
61                  iob_signal(
62                      name="wb_clk_i",
63                      descr="Clock input.",
64                  ),
65                  iob_signal(
66                      name="wb_rst_i",
67                      descr="Reset input.",
68                  ),
69                  iob_signal(
70                      name="wb_tgd_i",
71                      descr="Data tag type. Contains information
                            associated with data lines [dat] and [strb
                            ].",
72                  ),
73                  iob_signal(
74                      name="wb_tgd_o",
75                      descr="Data tag type. Contains information
                            associated with data lines [dat] and [strb
                            ].",
76                  ),
77                  iob_signal(
78                      name="wb_err_i",
79                      descr="Error input. Indicates abnormal cycle
                            termination.",
80                  ),
81                  iob_signal(
```



```
                    name="wb_lock_o",
                    descr="Lock output. Indicates current bus
                       cycle is uninterruptable.",
                ),
                iob_signal(
                    name="wb_rty_i",
                    descr="Retry input. Indicates interface is
                       not ready to accept or send data, and
                       cycle should be retried.",
                ),
                iob_signal(
                    name="wb_tga_o",
                    descr="Address tag type. Contains information
                        associated with address lines [adr], and
                       is qualified by signal [stb].",
                ),
                iob_signal(
                    name="wb_tgc_o",
                    descr="Cycle tag type. Contains information
                       associated with bus cycles, and is
                       qualified by signal [cyc].",
                ),
            ]
```

View Source

The Wishbone interface has the configurable widths:

- ADDR_W
- DATA_W

For example, to add an Wishbone port to a core, add the following python dictionary to the core's ports list:

```
{
    "name": "example_port_m",
    "descr": "Wishbone manager port",
    "signals": {
        "type": "wb",
        "ADDR_W": 32,
        "DATA_W": 32,
    },
},
```

IOb Native

The IOb Native interface is an open source interface developed by IObundle. It simplifies the connections between core components due to its reduced amount of signals when compared to other standard interfaces. The Py2HWSW core library 4.8 provides core's to convert between the IOb and other standard interfaces. A description of the IOb interface is available at: https://github.com/IObundle/py2hwsw/blob/main/py2hwsw/lib/hardware/buses/iob_tasks/README.md

The IOb interface is implemented by the 'iobInterface' class of the interfaces.py script.

```
class iobInterface(_interface):
    """Class to represent an IOb interface for generation"""

    # Widths for the IOb interface
    addr_w: str or int = 32
    data_w: str or int = 32
    strb_w: str or int = None

    def __post_init__(self):
        super().__post_init__()
        if not self.strb_w:
            self.strb_w = f"{self.data_w}/8"
        self.__set_signals()

    def __set_signals(self):
        """Set signals for the IOb interface."""

        self._signals = self._signals + [
            iob_signal(name="iob_valid_o", descr="Request address is valid."),
            iob_signal(name="iob_addr_o", width=self.addr_w, descr="Byte address."),
            iob_signal(name="iob_wdata_o", width=self.data_w, descr="Write data."),
            iob_signal(
                name="iob_wstrb_o", width=self.strb_w, descr="Write strobe."
            ),
            iob_signal(name="iob_rvalid_i", descr="Read data valid."),
            iob_signal(name="iob_rdata_i", width=self.data_w, descr="Read data."),
            iob_signal(name="iob_ready_i", descr="Interface ready."),
        ]
```

View Source

The IOb interface has the configurable widths:

- ADDR_W
- DATA_W

For example, to add an IOb port to a core, add the following python dictionary to the core's ports list:

```
{
    "name": "example_port_m",
    "descr": "IOb manager port",
    "signals": {
        "type": "iob",
        "ADDR_W": 32,
        "DATA_W": 32,
    },
},
```

4.7 Special cases

Most of the cores provided by the py2hwsw's library are built using the standard interfaces mentioned in section 3.4.

However, there are some cores that due to limitations of the standard interfaces, rely instead on internal py2hwsw methods for extra features. The following list describes the cores don't rely solely on the standard interfaces.

- **iob_system**: This core uses the 'is_system' attribute to enable an internal py2hwsw method that automatically fixes the address widths of the cbus interfaces of the system's peripherals.
- **iob_csrs**: The py2hwsw tool contains an internal method to automatically search for the "iob_csrs" subblock and insert a "<prefix>_cbus_s" port on the issuer core of this subblock. It then connects this newly created "<prefix>_cbus_s" port of the issuer core to the iob_csrs "control_if_s" port. The '<prefix>' is replaced by instance name of iob_csrs subblock.

4.8 Core library

The Py2HWSW framework includes a library of cores ready for use.

Name	Directory
iob_axistream_in	peripherals/iob_axistream_in
iob_regfileif	peripherals/iob_regfileif
iob_timer	peripherals/iob_timer
iob_macc	peripherals/iob_macc
iob_nco	peripherals/iob_nco
iob_axistream_out	peripherals/iob_axistream_out
iob_dma	peripherals/iob_dma
iob_gpio	peripherals/iob_gpio
iob_bootrom	peripherals/iob_bootrom
iob_uart	peripherals/iob_uart
iob_system	iob_system
iob_div_pipe	hardware/arith_logic/div/iob_div_pipe
iob_div_subshift	hardware/arith_logic/div/iob_div_subshift
iob_div_subshift_frac	hardware/arith_logic/div/iob_div_subshift_frac
iob_div_subshift_signed	hardware/arith_logic/div/iob_div_subshift_signed
iob_fp_round	hardware/arith_logic/iob_fp/iob_fp_round
iob_fp_float2uint	hardware/arith_logic/iob_fp/iob_fp_float2uint
iob_fp_add	hardware/arith_logic/iob_fp/iob_fp_add
iob_fp_div	hardware/arith_logic/iob_fp/iob_fp_div
iob_fp_uint2float	hardware/arith_logic/iob_fp/iob_fp_uint2float
iob_fp_int2float	hardware/arith_logic/iob_fp/iob_fp_int2float
iob_fp_cmp	hardware/arith_logic/iob_fp/iob_fp_cmp
iob_fp_mul	hardware/arith_logic/iob_fp/iob_fp_mul
iob_fp_float2int	hardware/arith_logic/iob_fp/iob_fp_float2int
iob_fp_sqrt	hardware/arith_logic/iob_fp/iob_fp_sqrt
iob_fp_dq	hardware/arith_logic/iob_fp/iob_fp_dq
iob_fp_fpu	hardware/arith_logic/iob_fp/iob_fp_fpu
iob_fp_minmax	hardware/arith_logic/iob_fp/iob_fp_minmax
iob_fp_special	hardware/arith_logic/iob_fp/iob_fp_special
iob_fp_clz	hardware/arith_logic/iob_fp/iob_fp_clz
iob_int_sqrt	hardware/arith_logic/iob_int_sqrt
iob_add2	hardware/arith_logic/iob_add2
iob_ctls	hardware/arith_logic/iob_ctls
iob_diff	hardware/arith_logic/iob_diff
iob_prio_enc	hardware/arith_logic/iob_prio_enc
iob_edge_detect	hardware/arith_logic/iob_edge_detect
iob_counter	hardware/arith_logic/counter/iob_counter
iob_counter_ld	hardware/arith_logic/counter/iob_counter_ld
iob_modcnt	hardware/arith_logic/counter/iob_modcnt
iob_xor	hardware/arith_logic/iob_xor
iob_functions	hardware/arith_logic/iob_functions
iob_add	hardware/arith_logic/iob_add
iob_acc	hardware/arith_logic/accumulators/iob_acc
iob_acc_ld	hardware/arith_logic/accumulators/iob_acc_ld

iob_clock	hardware/clocks_resets/iob_clock
iob_reset	hardware/clocks_resets/iob_reset
iob_pulse_gen	hardware/clocks_resets/iob_pulse_gen
iob_clkbuf	hardware/clocks_resets/iob_clkbuf
iob_clkmux	hardware/clocks_resets/iob_clkmux
iob_axis_s_axi_m_write	hardware/buses/iob_axis_s_axi_m_write
iob_mux	hardware/buses/iob_mux
iob_bus_width_converter	hardware/buses/iob_bus_width_converter
iob_axi_m_write_dma	hardware/buses/iob_axi_m/iob_axi_m_write_dma
iob_axi_m	hardware/buses/iob_axi_m/iob_axi_m
iob_axi_m_read	hardware/buses/iob_axi_m/iob_axi_m_read
iob_axi_m_dma	hardware/buses/iob_axi_m/iob_axi_m_dma
iob_axi_m_read_dma	hardware/buses/iob_axi_m/iob_axi_m_read_dma
iob_axi_m_write	hardware/buses/iob_axi_m/iob_axi_m_write
iob_demux	hardware/buses/iob_demux
iob_split	hardware/buses/iob_split
iob_universal_converter	hardware/buses/iob_universal_converter
iob_axis2fifo	hardware/buses/iob_axis2fifo
iob_fifo2axis	hardware/buses/iob_fifo2axis
iob_axi_interconnect_wrapper	hardware/buses/iob_axi_interconnect_wrapper
iob_axi_crossbar	hardware/buses/iob_axi_crossbar
iob_axis_m_axi_m_read	hardware/buses/iob_axis_m_axi_m_read
iob_axil2axi	hardware/buses/iob_axil2axi
iob_iob2axi	hardware/buses/iob_iob2axi
iob_iob2apb	hardware/buses/iob_iob2apb
iob_axi_interconnect	hardware/buses/iob_axi_interconnect
iob_tasks	hardware/buses/iob_tasks
iob_axis_tasks	hardware/buses/iob_axis_tasks
iob_axi_merge	hardware/buses/iob_axi_merge
iob_apb2iob	hardware/buses/iob_apb2iob
iob_axi2iob	hardware/buses/iob_axi2iob
iob_axi2axil	hardware/buses/iob_axi2axil
iob_bus_demux	hardware/buses/iob_bus_demux
iob_asym_converter_m_s	hardware/buses/iob_asym_converter_m_s
iob_iob_s_axi_m	hardware/buses/iob_iob_s_axi_m
iob_axi_split	hardware/buses/iob_axi_split
iob_reverse	hardware/buses/iob_reverse
iob_merge	hardware/buses/iob_merge
iob_axi_full_xbar	hardware/buses/iob_axi_full_xbar
iob_asym_converter	hardware/buses/iob_asym_converter
iob_axil_split	hardware/buses/iob_axil_split
iob_axis2ahb	hardware/buses/iob_axis2ahb
iob_axil2iob	hardware/buses/iob_axil2iob
iob_iob2wishbone	hardware/buses/iob_iob2wishbone
iob_iob2axil	hardware/buses/iob_iob2axil

iob_arbiter	hardware/buses/iob_arbiter
iob_wishbone2iob	hardware/buses/iob_wishbone2iob
iob_address_translator	hardware/buses/iob_address_translator
iob_axi_ram	hardware/memories/iob_axi_ram
iob_rom_sp	hardware/memories/rom/iob_rom_sp
iob_rom_2p	hardware/memories/rom/iob_rom_2p
iob_rom_tdp	hardware/memories/rom/iob_rom_tdp
iob_rom_atdp	hardware/memories/rom/iob_rom_atdp
iob_ram_2p	hardware/memories/ram/iob_ram_2p
iob_ram_sp	hardware/memories/ram/iob_ram_sp
iob_ram_atdp	hardware/memories/ram/iob_ram_atdp
iob_ram_tdp	hardware/memories/ram/iob_ram_tdp
iob_ram_t2p_tiled	hardware/memories/ram/iob_ram_t2p_tiled
iob_ram_sp_se	hardware/memories/ram/iob_ram_sp_se
iob_ram_sp_be	hardware/memories/ram/iob_ram_sp_be
iob_ram_t2p_be	hardware/memories/ram/iob_ram_t2p_be
iob_ram_tdp_be	hardware/memories/ram/iob_ram_tdp_be
iob_ram_tdp_be_xil	hardware/memories/ram/iob_ram_tdp_be_xil
iob_ram_at2p	hardware/memories/ram/iob_ram_at2p
iob_ram_atdp_be	hardware/memories/ram/iob_ram_atdp_be
iob_ram_t2p	hardware/memories/ram/iob_ram_t2p
iob_ahb_ram	hardware/memories/iob_ahb_ram
iob_memwrapper	hardware/memories/iob_memwrapper
iob_aoi	hardware/basic_tests/iob_aoi
iob_aoi	hardware/basic_tests/iob_aoi
iob_rom_acc	hardware/basic_tests/iob_rom_acc
iob_inv	hardware/basic_tests/iob_inv
iob_inv	hardware/basic_tests/iob_inv
iob_or	hardware/basic_tests/iob_or
iob_or	hardware/basic_tests/iob_or
iob_fsm3	hardware/basic_tests/iob_fsm3
iob_fsm_defaults	hardware/basic_tests/iob_fsm_defaults
iob_csrs_demo	hardware/basic_tests/iob_csrs_demo
iob_and	hardware/basic_tests/iob_and
iob_and	hardware/basic_tests/iob_and
iob_2to1mux	hardware/basic_tests/iob_2to1mux
iob_2to1mux	hardware/basic_tests/iob_2to1mux
iob_sync	hardware/synchronizers/iob_sync
iob_reset_sync	hardware/synchronizers/iob_reset_sync
iob_sync_reg	hardware/registers/iob_sync_reg
iob_reg	hardware/registers/iob_reg
iob_regarray_at2p	hardware/registers/regarray/iob_regarray_at2p
iob_regarray_sp	hardware/registers/regarray/iob_regarray_sp
iob_regarray_2p	hardware/registers/regarray/iob_regarray_2p
iob_regarray_dp_be	hardware/registers/regarray/iob_regarray_dp_be

iob_iobuf	hardware/iob_iobuf
iob_shift_reg	hardware/shifters/iob_shift_reg
iob_sipo_reg	hardware/shifters/iob_sipo_reg
iob_piso_reg	hardware/shifters/iob_piso_reg
iob_pack	hardware/shifters/iob_pack
iob_unpack	hardware/shifters/iob_unpack
iob_csrs	hardware/iob_csrs
iob_xilinx_iddr	hardware/amd/iob_xilinx_iddr
iob_xilinx_clock_wizard	hardware/amd/iob_xilinx_clock_wizard
iob_xilinx_ibufg	hardware/amd/iob_xilinx_ibufg
iob_xilinx_ddr4_ctrl	hardware/amd/iob_xilinx_ddr4_ctrl
iob_xilinx_axi_interconnect	hardware/amd/iob_xilinx_axi_interconnect
iob_xilinx_oddre1	hardware/amd/iob_xilinx_oddre1
iob_xilinx_oddr	hardware/amd/iob_xilinx_oddr
iob_gray_counter	hardware/fifo/iob_gray_counter
iob_fifo_async	hardware/fifo/iob_fifo_async
iob_fifo_sync	hardware/fifo/iob_fifo_sync
iob_bfifo	hardware/fifo/iob_bfifo
iob_gray2bin	hardware/fifo/iob_gray2bin
iob_altera_clk_buf_altclkctrl	hardware/altera/iob_altera_clk_buf_altclkctrl
iob_altddio_in	hardware/altera/iob_altddio_in
iob_altera_alt_ddr3	hardware/altera/iob_altera_alt_ddr3
iob_alt_iobuf	hardware/altera/iob_alt_iobuf
iob_altera_ddio_out_clkbuf	hardware/altera/iob_altera_ddio_out_clkbuf
iob_altddio_out	hardware/altera/iob_altddio_out
iob_printf	software/iob_printf
iob_coverage_analyze	software/iob_coverage_analyze
iob_linux_device_drivers	software/iob_linux_device_drivers
iob_str	software/iob_str

Table 3: Cores available in the library of the Py2HWSW framework. The *Directory* column is the path to the core's setup directory, relative to the Py2HWSW lib directory `py2hwsw/lib/`.

Table 3 lists the cores available in the Py2HWSW framework's core library.

Each core contains its own user guide, which can be built using the following commands:

```
py2hwsw <core_name> setup
make -C ../<core_name>_V<core_version>/ doc-build
xdg-open ../<core_name>_V<core_version>/document/ug.pdf
```

5 How To Use

5.1 Setup

To set up a core with Py2HWSW, you'll need to have Nix installed on your system. You can download and install Nix from the official Nix website. Once Nix is installed, you can clone the Py2HWSW repository with the following command:

```
git clone --recursive git@github.com:IObundle/py2hwsw.git
```

Next, navigate to the Py2HWSW directory and run the command `nix-shell` to enter the Nix-shell environment. This will ensure that all dependencies required by Py2HWSW are installed and available.

To set up a core, you can use the command:

```
nix-shell --run "py2hwsw $(CORE) setup --build_dir '$(BUILD_DIR)'
    --py_params 'param1=param1_val:param2=param2_val'"
```

This command will generate the necessary files and directories for your core in the specified build directory.

You can customize the setup process by passing additional options to the `py2hwsw` command. For example, you can disable format and linting checks by adding the options `-no_verilog_lint` and `-no_verilog_format` to the command.

Here's an example of a setup directory structure:

```
mycore/
  mycore.py
  hardware/
    src/
      mycore.v
```

In this example, the `mycore.py` file contains the core description, and the `hardware/src` directory contains the Verilog source files for the core.

In some cases, the Verilog source file (`mycore.v`) may not be necessary, as the `mycore.py` file can describe the entire core, including its ports, wires, components, and even custom Verilog code. This allows for a high degree of flexibility and customization, as users can define their core's architecture and behavior entirely within the Python description file.

The Python description is particularly useful because it enables the creation of higher-level abstractions, making it easier to design and work with complex hardware components. Additionally, the use of Python parameters allows for dynamic modification of cores, enabling users to easily customize and adapt their designs to different use cases and applications.

To set up this core, you would run the command:

```
nix-shell --run "py2hwsw mycore setup --build_dir './build' --
    py_params 'param1=param1_val:param2=param2_val'"
```

This would generate the necessary files and directories for the core in the `./build` directory.

Note that you can customize the setup process to fit your specific needs by modifying the core description, Verilog source files, and setup command options.

5.2 Simulation

To simulate a core using Py2HWSW, you can use the `make sim-run` command inside the generated build directory. This command will run the simulation using the default simulator (Icarus Verilog). You can specify the simulator to be used using the `SIMULATOR` variable.

For example, to simulate the core using Verilator, you can run the command `make sim-run SIMULATOR=verilator` inside the core's build directory. This will compile the testbench and run the simulation, displaying the output on the console.

Py2HWSW also provides a universal Verilator testbench that can be used to simulate IP cores. The testbench behaves like a processor reading and writing to the core's control and status registers (CSRs), allowing for easy testing and verification of the core's functionality.

You can customize the simulation process by modifying the testbench and simulation parameters, such as the simulation time, input stimuli, and output signals to be monitored. Additionally, you can use other simulators, such as VCS or QuestaSim, by specifying the corresponding simulator variable.

Some cores in the Py2HWSW library also include a tester that can be used to verify their functionality. Examples of such cores include `iob_aoi`, `iob_pulse_gen`, and `iob_system`. These testers can be run along with the core to test its behavior and ensure that it is working as expected.

To run the tester, simply navigate to the tester's build directory, usually located inside the core's build directory in a folder named `tester/`, and run the command `make sim-run`. This will compile and run the tester, allowing you to verify the core's functionality and debug any issues that may arise. By providing these testers, Py2HWSW makes it easier to develop and test complex hardware components, and ensures that the cores in the library are reliable and functional.

5.3 ASIC Synthesis

To synthesize a core for an Application-Specific Integrated Circuit (ASIC) using Py2HWSW, you can use the `make syn-build` command in the generated build directory. This command will run the ASIC synthesis tool, Cadence Genus or Yosys, to generate a netlist for the specified ASIC process.

Before running the `make syn-build` command, you'll need to specify the ASIC synthesizer that you want to use. You can do this by setting the `SYNTHESIZER` environment variable or by passing it as a command-line argument. The supported values are `yosys` (the default) and `genus`. The target ASIC process is selected with the `NODE` variable, which defaults to `umc130`.

For example, to synthesize the core with yosys, you can run the command `make syn-build SYNTHESIZER=yosys`. This will generate a netlist that can be used as input for further ASIC design and verification tools, such as place and route, static timing analysis, and physical verification.

Py2HWSW supports a range of ASIC processes and libraries, and provides a set of pre-configured process files that make it easy to get started with ASIC synthesis. By using Py2HWSW to synthesize your cores, you can take advantage of the high performance and low power consumption of ASICs, while minimizing the complexity and effort required to develop and deploy your designs. The synthesis tools and the supported technology nodes are in the subdirectories of the Py2HWSW repository syn folder https://github.com/IObundle/py2hwsw/tree/main/py2hwsw/hardware/syn: `genus` and `yosys` hold the tool scripts, while `umc130` and the `tsmc_*` directories hold the process setup files.

5.4 FPGA Compilation

To compile a core for an FPGA using Py2HWSW, you can use the `make fpga-build` command in the generated build directory. This command will run the FPGA synthesis and implementation tools, such as Vivado or Quartus, to generate a bitstream for the specified FPGA board.

Before running the `make fpga-build` command, you'll need to specify the FPGA board that you want to target. You can do this by setting the `BOARD` environment variable or by passing it as a command-line argument.

For example, to compile the core for a Xilinx Artix-7 Basys 3 FPGA, you can run the command `make fpga-build BOARD=iob_basys3`. This will generate a bitstream that can be loaded onto the FPGA using the appropriate programming tools.

Py2HWSW supports a range of FPGA boards and devices, and provides a set of pre-configured board files that make it easy to get started with FPGA development. By using Py2HWSW to compile and implement your cores, you can take advantage of the flexibility and performance of FPGAs, while minimizing the complexity and effort required to develop

and deploy your designs. The list of available FPGA boards can be found in the `vivado` and `quartus` subdirectories of the Py2HWSW repository fpga folder https://github.com/IObundle/py2hwsw/tree/main/py2hwsw/hardware/fpga.

5.5 End to End Examples

This section provides three end-to-end examples of using Py2HWSW to generate and verify digital hardware cores. The examples cover the iob_aoi, iob_pulse_gen, and iob_soc cores, showcasing the automation of Verilog module generation from attributes.

5.5.1 iob_aoi Example

The iob_aoi core is a simple example that combines an AND, OR, and invert logic gates. The core's attributes are defined in the iob_aoi.py file, available at https://github.com/IObundle/py2hwsw/tree/main/py2hwsw/lib/hardware/basic_tests/iob_aoi. To generate the iob_aoi core, follow these steps:

1. Create or modify the iob_aoi.py file to set the attributes of the core, describing how it should be generated using the Py2HWSW standard core dictionary interface.
2. Optionally, add more files to the setup directory as needed, such as manual Verilog sources or templates, scripts, or software.
3. Call the Py2HWSW setup process using the command:

```
nix-shell --run "py2hwsw iob_aoi setup --build_dir '$(
   BUILD_DIR)'"
```

4. The generated build directory contains all the Verilog sources, Makefile, and configurations to run the core in various flows (simulation, FPGA).
5. To run the core in simulation, call the command: `make sim-run` from the build directory.

5.5.2 iob_pulse_gen Example

The iob_pulse_gen core is used to generate signal pulses with configurable start and duration. The core's attributes are defined in the iob_pulse_gen.py file, available at https://github.com/IObundle/py2hwsw/blob/main/py2hwsw/lib/hardware/clocks_resets/iob_pulse_gen/iob_pulse_gen.py. To generate the iob_pulse_gen core, follow these steps:

1. Create or modify the iob_pulse_gen.py file to set the attributes of the core, describing how it should be generated using the Py2HWSW standard core dictionary interface.

2. Optionally, add more files to the setup directory as needed, such as manual Verilog sources or templates, scripts, or software.
3. Call the Py2HWSW setup process using the command:

```
nix-shell --run "py2hwsw iob_pulse_gen setup --build_dir '$(BUILD_DIR)'"
```

4. The generated build directory contains all the Verilog sources, Makefile, and configurations to run the core in various flows (simulation, FPGA).
5. To run the core in simulation, call the command: `make sim-run` from the build directory.

5.5.3 iob_soc Example

The iob_soc core is a more complex example used to create a system on chip. The core's attributes are defined in the iob_soc.py file, available at https://github.com/IObundle/iob-soc. The iob_soc.py file supports high-level Python parameters that allow configuring main SoC components like the CPU, memories, and peripherals. To generate the iob_soc core, follow these steps:

1. Create or modify the iob_soc.py file to set the attributes of the core, describing how it should be generated using the Py2HWSW standard core dictionary interface.
2. Optionally, add more files to the setup directory as needed, such as manual Verilog sources or templates, scripts, or software.
3. Call the Py2HWSW setup process using the command:

```
nix-shell --run "py2hwsw iob_soc setup --build_dir '$(BUILD_DIR)'"
```

4. The generated build directory contains all the Verilog sources, Makefile, and configurations to run the core in various flows (simulation, FPGA).
5. To run the core in simulation, call the command: `make sim-run` from the build directory.

These examples demonstrate the automation of Verilog module generation from attributes using Py2HWSW, showcasing the flexibility and ease of use of the framework.

5.6 Customizing Py2HWSW

Py2HWSW allows users to customize its behavior and core generation process. When running cores directly from the cloned Py2HWSW repository, users can modify the cores or Py2HWSW scripts for debugging purposes.

The Py2HWSW repository contains several main folders, including lib, scripts, and generic folders.


```
.
└── py2hwsw
    ├── <py2 generic folders>
    ├── lib
    └── scripts
```

The py2hwsw folder contains generic folders that are copied to every core build directory set up via Py2HWSW. These folders include standard Makefiles, FPGA board constraints, simulator dependencies, and other essential files.

To override the default files, users can create a file with the same name in their core's setup directory. For example, to override the default py2hwsw/document/tsrc/sim_desc.tex, create a new document/tsrc/sim_desc.tex in the core's setup directory. Py2HWSW will first copy the generic default file to the build directory and then copy the core files from the setup directory, overriding the default file.

The lib directory contains a library of cores provided by Py2HWSW. These cores are intended to be bug-free and do not typically require modifications. However, if users need to modify a core for their project, they can copy the corresponding core's setup directory to a subfolder in their project directory. Py2HWSW will use the first core it finds with the required name, so users can create custom modifications to the core specific to their project.

For example, to modify the iob_and core, copy its folder from the Py2HWSW repository and place it in the user's project directory. When calling Py2HWSW from the project directory, it will find the copied iob_and folder first and use it to generate the iob_and core.

The scripts folder contains the Python scripts that make up the Py2HWSW tool. These scripts are typically only modified by developers, as they directly change the Py2HWSW program's behavior. However, users can modify these scripts for quick bug fixes or to add custom functionality.

By customizing Py2HWSW, users can tailor the tool to their specific needs and create custom cores and workflows. This flexibility allows users to adapt Py2HWSW to their project's requirements and create complex digital designs with ease.

5.7 Troubleshooting

When encountering errors during the setup process with Py2HWSW, there are several steps you can take to diagnose and resolve the issue.

5.7.1 Error Messages

The main error message is usually printed in red color and provides information on where the issue originates, often due to a misconfiguration in the provided core dictionary. The

traceback that follows is more useful for Py2HWSW developers, as it contains information on which Py2HWSW function has thrown the error.

5.7.2 Debugging Options

If more information is required to troubleshoot the issue, you can use the following options:

- The –debug_level flag: When calling py2hwsw with the –debug_level flag, you can print debug messages during the setup process. The higher the debug level, the more messages are printed.
- Adding print statements: You can add print statements in your own core's .py file to understand when the script is being called and what contents it contains.
- Modifying Py2HWSW scripts: Adding print statements to the Py2HWSW main scripts can also be useful, but this requires understanding the inner workings of Py2HWSW and is usually reserved for developers.

5.7.3 Build Process Errors

If the Py2HWSW setup process completes successfully, but the build process for a flow from the build directory gives errors (e.g., calling the Makefile from the build directory for simulation), follow these steps:

1. Check the generated Verilog sources: Verify that the contents of the generated Verilog sources are as intended.
2. Check tool-specific files: If the error message is simulator/tool-specific and does not seem related to the Verilog sources, check the constraints files, Makefiles, and other tool-specific files to determine where the issue originates.

5.7.4 Overriding Py2HWSW Generated Files

If you need to modify a Py2HWSW generated file, you can override it by creating a new file with the same name in your core's setup directory. This allows you to customize the generated files to suit your specific needs.

By following these troubleshooting steps, you should be able to identify and resolve issues that arise during the setup and build process with Py2HWSW.

